The ARC-AGI-3 Lottery: a testable model of LLM-agent scores under Kaggle compute

Shreekumar Shah (Kaggle: shreekumarshah). November 2026. Code, data and sealed files: github.com/Shreekumar-Shah-AICTE/arc-agi-3-lottery

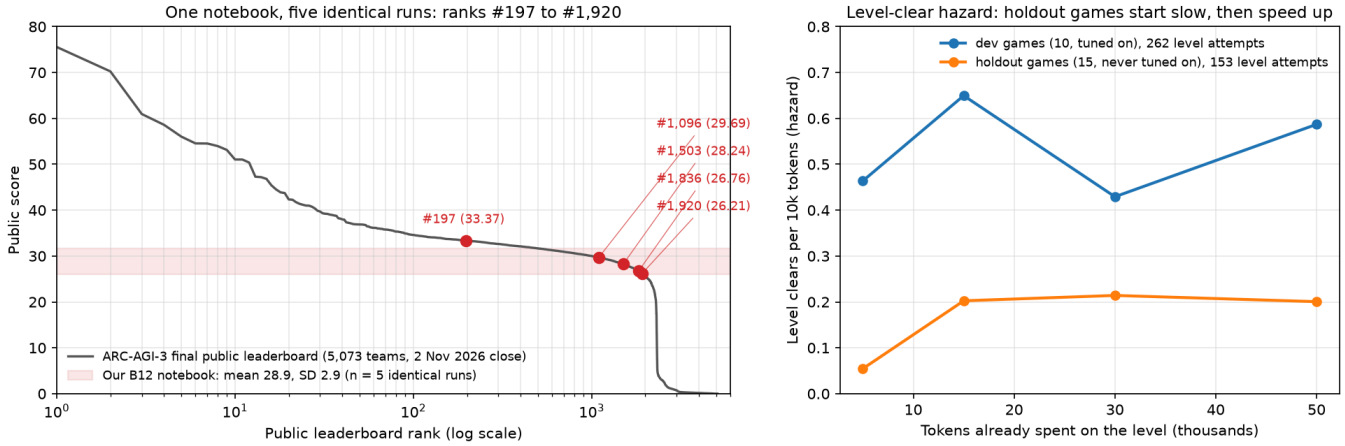
Abstract

We present a testable model of what sets an LLM agent's ARC-AGI-3 score under fixed Kaggle compute, and of which changes can beat its run-to-run noise. Byte-identical code submitted five times placed between 197th and 1,920th of 5,073 teams. In the model, each level waits for a "crack" (the moment the agent finds what the level wants), measured in generated tokens: with probability q the level is a wall within the budget, otherwise the wait is lognormal. Fitted on four control runs, the model reproduces the run-to-run noise (SD 5.6 against 5.3 levels) only if level difficulty and walls persist across runs. The walls sit in the 15 games we did not develop on, where even unlimited play would reach only about 56% to 69% of the score (73% to 81% over all 25 public games). So a change beats the noise only if it adds cracks per GPU-second, and walls, not time, set the distance to 85%. Five predictions were sealed in advance. Doubling the play time was predicted to clear 127 levels (90% interval 110 to 145) and cleared 141: inside the interval but 14 above the point prediction, about half of the gap traced later to the harness's endgame. A guard that keeps games on their first level from being shelved passed in one partial run, a token-supply cut went unscored, and the private ARC-AGI-3 and ARC-AGI-2 boards are pending. On ARC-AGI-2, best-of-n draws of one public notebook lineage account for about 70% of the rise of a 2,133-team plateau's mean with the number of entries, and run luck of unchanged code for about half; on 120 public SWE-Bench-Verified runs, per-task luck explains most of the noise. We give run counts and a two-number comparison that needed about 5 times fewer runs on our data.

1. Introduction

ARC-AGI asks whether a system can learn new skills efficiently, from little experience, the way people do [Chollet19]. ARC-AGI-3 makes the test interactive. An agent plays games it has never seen, with no instructions, and is scored on the levels it completes and on how many actions it needs compared with people [ARC3]. The Paper Prize asks how much a paper raises the chance of anyone reaching 85% on ARC-AGI [ARCPaper26]; on ARC-AGI-3 the grand prize needs 100% [ARC3Kaggle]. Progress toward either line is read off leaderboards, so whatever moves a leaderboard decides what the field learns.

Our question started with a resubmission. We submitted the same notebook, byte for byte, five times to the ARC-AGI-3 Kaggle leaderboard. It scored 33.37, 29.69, 28.24, 26.76 and 26.21 (SD 2.87). On the final public board of 5,073 teams, those five draws place 197th, 1,096th, 1,503rd, 1,836th and 1,920th (Fig. 1; ties counted against us). Over the following weeks we ran dozens of experiments on the agent. Thirty-two ended as killed, noise, failed, parked or invalid (Appendix A), and nothing we tried after this notebook beat its run-to-run noise (with one leaderboard draw per arm, a gain needs about 11.4 points to be detected reliably; Section 4.6).



Sources: Kaggle public leaderboard and our 5 scored runs of arc3-b12-x6dprime v1 (Kaggle CLI; final board fetched 4 Nov 2026); 415 level attempts from 9 commit runs (repo analysis/s7). Hazard bins 0-10k, 10-20k, 20-40k, 40-60k tokens.

Fig. 1. Left: five byte-identical runs of our B12 notebook placed on the final public ARC-AGI-3 board. Right: level-clear hazard in token time (dev vs holdout games).

This paper asks what sets an LLM agent's ARC-AGI-3 score under fixed compute, and what follows for progress and for evaluation. Our answer is a small model. Each level of each game waits for a "crack", the moment the agent finds what the level wants, and we measure the wait in generated tokens. With probability q the level is a wall that the agent will not crack within the budget; otherwise the wait is lognormal. Three numbers fitted on four control runs describe the agent, though they pool two kinds of games: the 10 we developed on show no walls, and the 15 we held out carry all of them (Section 4.3). Level difficulty that persists from run to run explains why identical runs differ by about 5 levels rather than 12. The token budget then turns the model into a score. What is new is the pairing of a mechanism with sealed tests: a model of where an LLM agent's leaderboard noise comes from, checked by predictions time-stamped before their data existed, two of them for private leaderboards that are not yet published.

Contributions:

1. A mechanism (Section 3): the crack model and its link to the official score, with checks against held-out budgets and against other teams' runs.
2. The cause of the noise (Section 4.3): luck in which near-the-edge levels a run clears before its budget ends, on top of level difficulty and walls that do not change between runs. About half of the per-game noise is whether a game clears its first level before the harness's slot rule shelves it.
3. Sealed predictions (Section 4.4): for the private ARC-AGI-3 and ARC-AGI-2 leaderboards (pending until the hosts publish them); for doubling the play time (141 levels against 127 predicted: inside the 90% interval but near its top; in a simulation made after the result, about half of the excess comes from an endgame rule in the harness that shaped the control runs the sealed calculation continued); for a cut in token supply (unscored, because both slower runs left games unplayed); and for a guard that keeps games on level 1 from being shelved (passed in one partial run: 1 zero-level game among the 23 that started, against 1 to 4 in seven reference runs, but 2 games never started).
4. Tools for anyone comparing agents (Sections 4.5-4.6): two numbers per agent that needed about 5 times fewer runs than raw level totals on our data (at least 2.8 times), run counts for leaderboard comparisons, and our 32 negative results sorted by what each change did to the run.
5. A similar pattern on ARC-AGI-2 (Section 4.8): 2,133 teams share a plateau whose mean score rises with their number of entries; best of n draws of one notebook lineage accounts for about 70% of the rise, run luck alone for about half. On the 120 public SWE-Bench-Verified runs of Bjarnason et al., per-task luck likewise accounts for the run-to-run noise (86% pooled; exploratory).

Why this matters for the 85% goal: if most differences between similar agents are luck, a team that tunes against the leaderboard learns noise, and a field that reads leaderboards learns less than it thinks. The model says which kind of change can beat the noise (more cracks per GPU-second, fewer walls) and how many runs it takes to know. The ARC Prize 2024 report describes the paper awards as meant "to reward novel concepts regardless of how their solutions

scored" [ARC24]. Our own score is modest (best draw 33.37); our aim is to explain why scores like ours move the way they do.

2. Prior work

Noise in machine-learning evaluation. Run-to-run variation is often as large as the differences that papers report. In deep reinforcement learning the random seed alone can change a conclusion [Henderson18], which led to power calculations for the number of seeds [Colas18] and to interval estimates for the few-run regime [Agarwal21]. Benchmark variance has known sources, such as data sampling, initialization and hyperparameter choice [Bouthillier21]. For language models, seed variance has been measured across common benchmarks [Madaan24] and tied to a benchmark's signal-to-noise ratio [Heineman25]. Authors are asked to report the expected best score as a function of the number of trials [Dodge19], and to use clustered standard errors and question-level paired comparisons [Miller24]. For agents, single-run pass@1 on 60,000 SWE-Bench-Verified trajectories shifts by 2.2 to 6.0 points with the run chosen [Bjarnason26]; rerunning the same configuration explains about 54% of the outcome variance on scientific-agent tasks [Wiedmann26]; and leaderboard variance splits into signal and noise across 22 agent benchmarks [Hardy26].

We share this advice: report the spread and the number of runs. We differ in three ways. First, we give a mechanism, not only a measurement: a waiting-time model with three fitted numbers predicts how large the run-to-run noise is and how it changes with the budget. Second, we test it with predictions sealed before their data existed. Third, the setting is a hidden-test competition. A team sees one noisy draw per day and cannot inspect the test, so the noise is part of the score, not a reporting choice.

Leaderboards and resubmission. A public leaderboard reports a team's best entry, so it computes Dodge et al.'s quantity with n equal to the team's number of entries. Across over one hundred Kaggle competitions, public and final rankings show little evidence of substantial overfitting from test-set reuse [Roelofs19]. A leaderboard can limit what repeated submission learns [BlumHardt15], and a holdout set can be reused across many adaptive analyses without overfitting to it [Dwork15]. On Chatbot Arena, providers who test many private variants and publish only the best one gain rank through that selection [Singh25]. Our case differs in kind. The noise sits in the run, not in the split, so selecting the best public entry selects luck that will not carry over to the private half. On ARC-AGI-2, the plateau of 2,133 teams rises with the number of entries much as the best of n draws of one notebook would (Section 4.8).

Repeated sampling and test-time compute. Coverage grows steadily with the number of samples [Brown24], and the power law of that growth traces to a heavy-tailed distribution of per-problem success rates [Schaeffer25]. The crack model works the same way in token time, with fixed level difficulty in place of problem difficulty. A fixed test-time budget can be spent well or badly [Snell24], resampling against an imperfect verifier has limits [Stroebl24], and pass^k exposes the gap between an agent's best and its reliable behaviour [Yao24]. These works treat sampling as part of the method. On Kaggle, two kinds of resampling happen outside it. Inside a run, more generated tokens act like extra samples (Section 4.4 doubles the play time); across runs, the leaderboard resamples whole submissions. The crack model makes the first explicit, and the best-of-n view the second.

Waiting-time models. The closest ancestor of the crack model is the run-time distribution of a Las Vegas algorithm, by which stochastic search can be evaluated [HoosStutzle98]. Randomized backtracking search has heavy-tailed solve times that random restarts remove [Gomes00], and a restart schedule exists within a logarithmic factor of the best one when the distribution is unknown [Luby93]; we did not test whether restarting a long attempt on a level helps an LLM agent. Our wall share is the cured fraction of a mixture cure model, in which some units never have the event during follow-up [BerksonGage52] [Farewell82]; we apply it in generated-token time, with censoring at the end of each run. Telling a cured unit from a late event is a known difficulty of these models [MallerZhou96]. For us it is the question of whether a wall is a very slow crack (Section 4.3).

Comparing agents with fewer runs. Agents must be compared at equal cost [KapoorAgents24]. We describe each agent by two numbers: how many tokens it can generate in the budget (supply) and how many it needs per level (crack cost). Comparing crack cost on the same levels is a blocked design, in which level effects cancel as question effects do in paired comparisons of evals [Miller24], much as item response models let tinyBenchmarks evaluate language models on far fewer items [Polo24]: most of the variance in crack times belongs to the level, not to the run (Section 3.4). It needed

several times fewer runs than raw level counts (about 5 times on our data, at least 2.8; Section 4.6). METR summarizes an agent by the task length, in human working time, that it completes half the time [Kwa25]. Our crack cost is the agent-side counterpart, and its curve also shows where more budget stops buying (the walls).

Preregistration. Predictions time-stamped before the data exist separate tests from post-hoc stories [Nosek18]. Preregistration is not new to machine learning (a NeurIPS 2020 workshop was devoted to it [PreregML20]). We sealed ours with SHA-256 hashes, OpenTimestamps proofs anchored in Bitcoin, and a public Kaggle post; what we add is its use on a live competition's hidden leaderboard.

ARC. Chollet defines intelligence as skill-acquisition efficiency [Chollet19]. ARC-AGI-3 makes the test interactive and scores the actions an agent needs against a human baseline, level by level [ARC3]. The hosts qualified every game so that random play cannot beat its non-tutorial levels: random sweeps of one million steps, and a probability of a random win below 1 in 10,000 per level [ARC3, Section 3.5.1]. The luck we measure is therefore not random play but the question of when a capable agent's search finds a level's key. Several ARC methods search over samples: Greenblatt sampled about 8,000 programs per task and gained about 3 points of accuracy per doubling of samples [Greenblatt24], and product-of-experts scoring [Franzen25] and self-improving evolutionary program search [SOAR] also rank sampled candidates. Others change the learner, with induction and transduction models trained on synthetic tasks [Li24] or test-time training [Akyurek24]. Compute alone also moves ARC scores: on ARC-AGI-1, OpenAI's o3 preview scored 75.7% of the semi-private tasks within the leaderboard's compute limit and 87.5% with 172 times the compute [ARCo3]. On ARC-AGI-3, frontier systems outside Kaggle's limits score far higher: GPT-6 Astra reached 62.7% of the semi-private set with the hosts' standard harness and 99.9% with a harness that keeps the model's reasoning state between requests [ARCAstra26]. The hosts report a 'binary-like pattern': once a frontier model understands a game's mechanics, it plays within the range of human action efficiency [ARCAstra26]. That matches the crack model's premise that a level is a wait for one insight, followed by play. Where those papers improve the search or the learner, we ask what a fixed compute budget does to the score of any such search, and what that implies for reading leaderboards. Competitors already knew that resubmitting helps: a forum post describes a team turning identical code into 38.85 over about 100 submissions (Kaggle discussion 755320). We put a number on it: for our notebook, the best of 100 identical submissions is expected to score 7.2 points above a single one (Section 4.6).

3. Approach

3.1 Setting and data

ARC-AGI-3 on Kaggle hides 110 games. Half of them set the public leaderboard and half the private one. A submission runs on one RTX Pro 6000 GPU within a 9-hour limit. Our agent starts from dfranzen's public Milestone 2 notebook, which is built on Tufa Labs' Duck harness (Jeroen Cottaar). We added a ledger of solved levels: the rule the model wrote for each solved level stays in the prompt for the whole game, even after older history is dropped (X6). In a simple form it resembles the skill library of Voyager [Wang23] and the stored self-reflections of Reflexion [Shinn23]. We also took the slot-priority formula from shiiin9's public notebook (Form D1), which decides which games get the 10 generation streams and favours games that have reached later levels (D'). Together they make the B12 notebook. The language model is Qwen3.8-Flash-Next (4-bit weights) with a multi-token-prediction draft model, served by SGLang with 10 parallel streams.

We use four kinds of data. (1) Our Kaggle submissions: 17 on ARC-AGI-3 and 10 on ARC-AGI-2 [ARC2], with their leaderboard scores (Tables 1 and 2). (2) 39 development runs of our agents on the public games (1,022 scheduled game runs, counting games the scheduler never started; Appendix B, whose last five rows are this paper's own test runs from Section 4.4), each with a per-action log of generated tokens. One of them (f1_forklab) replays 32 saved game states in 256 short forks instead of playing whole games. In three of these runs (and in three of the five test runs) the stream scheduler never admitted one or two of the 25 games; we call the other runs valid, and every run-level number uses valid runs only. The standard control is FB25: all 25 games at once, up to 121 minutes per game, 10 streams, one pass. That gives each game 48.4 stream-minutes, the same as a real submission (10 streams x 532 minutes / 110 games). (3) Six public runs by other teams, whose logs we parsed the same way. (4) The final public leaderboards. We report the

development runs as measurements of noise and of the model, never as performance claims; performance claims use the Kaggle leaderboard only.

3.2 The crack model

An agent plays a game level by level. On each level it spends generated tokens (thinking and acting) until it finds what the level wants and clears it. We call the clear a crack and measure the wait T in generated tokens, not in actions or seconds, because tokens are what the budget buys.

The model has three numbers. With probability q a level is a wall: the agent will not crack it within any budget we can give it. Otherwise the wait is lognormal, log T ~ Normal(mu, s^2). A run ends when the budget ends, so the last attempt in each game is right-censored at the tokens it had spent. This is a mixture cure model [Farewell82] in token time. We fit it by maximum likelihood to the 480 level attempts of the four FB25 control runs (411 clears, 69 open at the end): q = 0.06, median wait exp(mu) = 22.1k tokens, s = 1.05. These pooled numbers average two populations (Section 4.3): the 10 games we developed on show no walls (q 0.000, median 16.9k), while the 15 held-out games carry all of them (q 0.110, median 35.7k; q 0.070 on all eight runs). The held-out fit is the better guide to the hidden games, so Section 5 gives the caps for both.

The model turns a budget into levels. A game with token budget B clears level after level while the summed waits stay below B. Games share the server, so a game that finishes early hands its share of the streams to the games still playing. Our simulator shares the tokens of finished games equally among the games still playing, an approximation of the real scheduler; Appendix D adds a simulation of the harness's slot rule itself.

3.3 From levels to score

ARC-AGI-3 scores a completed level by action efficiency against a human baseline h: min(1.15, (h/a)^2) for a actions. A game's score is the smaller of two weighted averages with weights w_l = l: the share of levels completed and the mean level score. The total is the mean over games [ARC3, eq. 1-3]. On the 13 valid runs of our main agent family the efficiency term costs 0.9% to 8.2% of the score compared with completion alone (median 5.0%). The reason is that on the levels it clears the agent usually needs fewer actions than the human baseline. Here a counts every action logged on the level, resets included, and h is the human baseline the scorer uses for that level. Over the 1,353 levels these runs cleared on all 25 games, a/h has a median of 0.65 (quartiles 0.44 to 0.93); 75% of the cleared levels sit at the 1.15 cap and 19% took more actions than the baseline. The completion term therefore decides most game scores: the score is set mostly by how many levels each game clears before its budget runs out. Later levels weigh more, so the levels that a run reaches only with luck are also the ones that count most.

3.4 Where the noise comes from

The same three numbers drawn fresh for every level and run (model H0) give a run-to-run SD of about 12 levels, far above the 5.3 we observe across six same-agent FB25 runs. The data say why. Waits are mostly a property of the level: across runs of one agent, the between-level SD of log wait is 0.80 and the within-level (run luck) SD is 0.49, so 73% of the variance belongs to the level. If level difficulty is fixed and walls are drawn fresh each run (H1), the SD is still too high. If both are fixed, so that a wall stays a wall in every run (H2), the simulated SD is 5.6 against 5.3 observed. Each run therefore holds the same loaded tickets: the hard levels and the walls do not move, and the luck is in how many near-the-edge levels a run happens to clear before its budget ends.

3.5 Two numbers per agent

To compare two agents we fit, on the same levels, log T(run r, level l) = a_l + b_r + e with e ~ Normal(0, s^2) and the open attempts censored (a lognormal accelerated-failure-time model with level effects, fitted by EM; s = 0.55). Each run then has a crack cost b_r, the log multiplier of tokens needed per level relative to the control, and a token supply S_r, the tokens it generated. Their combination m_r = (S_r / S_ctl) exp(-b_r) is an effective budget, which we convert to levels with the control's own budget curve. Because a_l absorbs the level's difficulty, b is far less noisy than a level count: its per-run standard error is 0.06-0.07.

3.6 Sealed predictions

Each prediction was written as a text file with its decision rule, hashed with SHA-256, time-stamped with OpenTimestamps (anchored in Bitcoin, so anyone can check the time with the proof file), and committed before its outcome was known; the hashes of both private-leaderboard predictions were also posted on Kaggle (discussion 755123: the ARC-AGI-3 hash in the post, the ARC-AGI-2 hash in a comment). The scorers (score_sealed.py, v5_readout.py, v10_readout.py, v11_readout.py, score_v9.py) were written before the outcomes. Sealed files are never edited; any later thought goes into a separate, dated note.

3.7 How this work was done

This work was carried out with an AI agent, starting on 30 September 2026. The agent wrote the analysis code, ran the experiments and drafted the text. The author chose the research questions, approved every GPU run and leaderboard submission, made the final leaderboard selections, published the notebook and posted the sealed hash. The repository keeps the written record of the work: a decision log, a ledger of 162 experiment rows (157 agent experiments plus this paper's five test runs), the sealed predictions with their hashes, and a registry that ties each number in this paper to the script output it comes from.

4. Results

4.1 Leaderboard

Table 1 lists all 17 of our ARC-AGI-3 submissions. The linked notebook, arc3-b12-x6dprime (public), is B12. Its first submission scored 33.37, our best, which places 197th of 5,073 on the final public board. Four byte-identical resubmissions scored 29.69, 28.24, 26.76 and 26.21 (mean of the five 28.85). The seven later submissions of the B12 harness (six variants, B30 twice), whether they changed the agent or only its start-up handling, scored 23.82 to 29.88 (mean 25.89); none beat B12. Nor can we show that B12 beats the notebook it started from. dfranzen's Milestone 2 notebook (B1), submitted twice, scored 28.30 and 26.72 (mean 27.51). B12's five draws are 1.34 points higher on average, with a standard error of 2.40 at the per-submission SD of 2.87 ($z = 0.56$, $p = 0.58$); detecting a true gain of that size with power 0.8 would take about 72 submissions per arm. Our own method (Section 3.5; exploratory) agrees: on the levels of the 10 games that B1's two development runs played, the seven valid 121-minute B12 runs need the same tokens per crack as B1 ($b = +0.013$, a factor of 1.01; bootstrap over runs, 90% interval -0.069 to +0.092). B12's changes moved tokens between games; they did not make cracks cheaper. A fork of a different public notebook (A1) scored 11.05. The private scores are not yet published; Section 4.4 gives our sealed prediction for them.

4.2 Does the model describe the runs?

Two checks use data that the fit did not see. First, a fit on the first half of each control run predicts the full-run level totals: 101.1 on average against 102.75 observed. Second, the same B12 notebook at one third of the budget (runs b12d25 and b12d25_r2 in Appendix B) cleared 43 and 45 levels; cutting the four full-budget runs at their token count per game gives 43, 36, 41 and 38 (mean 39.5). The token budget alone thus accounts for about 90% of the short runs' level count. Both short runs started only 17 of the 25 games; the cut uses the short runs' mean tokens per game, so a game that neither short run started counts as zero on both sides. Two more checks are in-sample and show only that the model's form fits. Fitted separately to each of six other teams' public runs, the model reproduces that run's level total within 1.3 levels (Section 4.8). The fitted curve matches the survival of open attempts in token time: at 62k tokens the observed share of attempts still open is 0.220 against 0.213 from the model (Fig. 2).

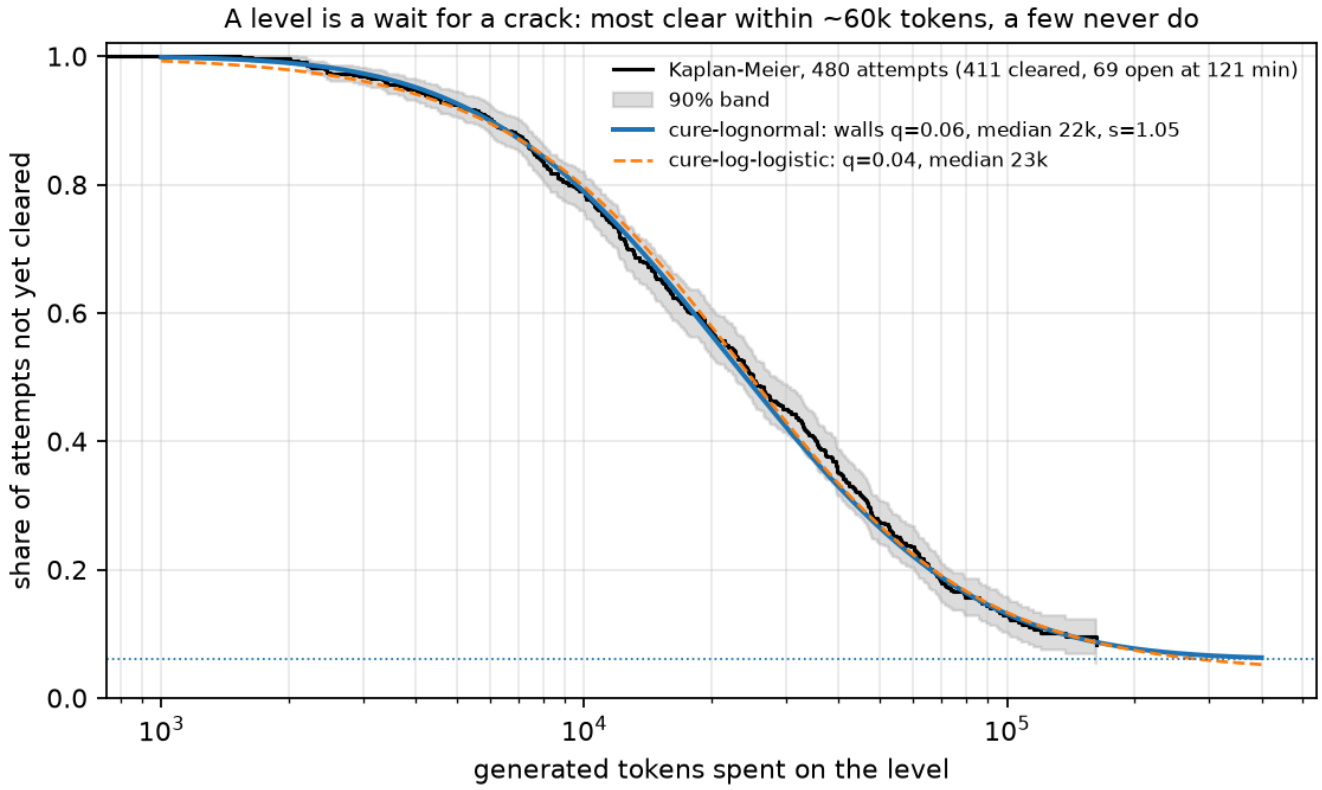
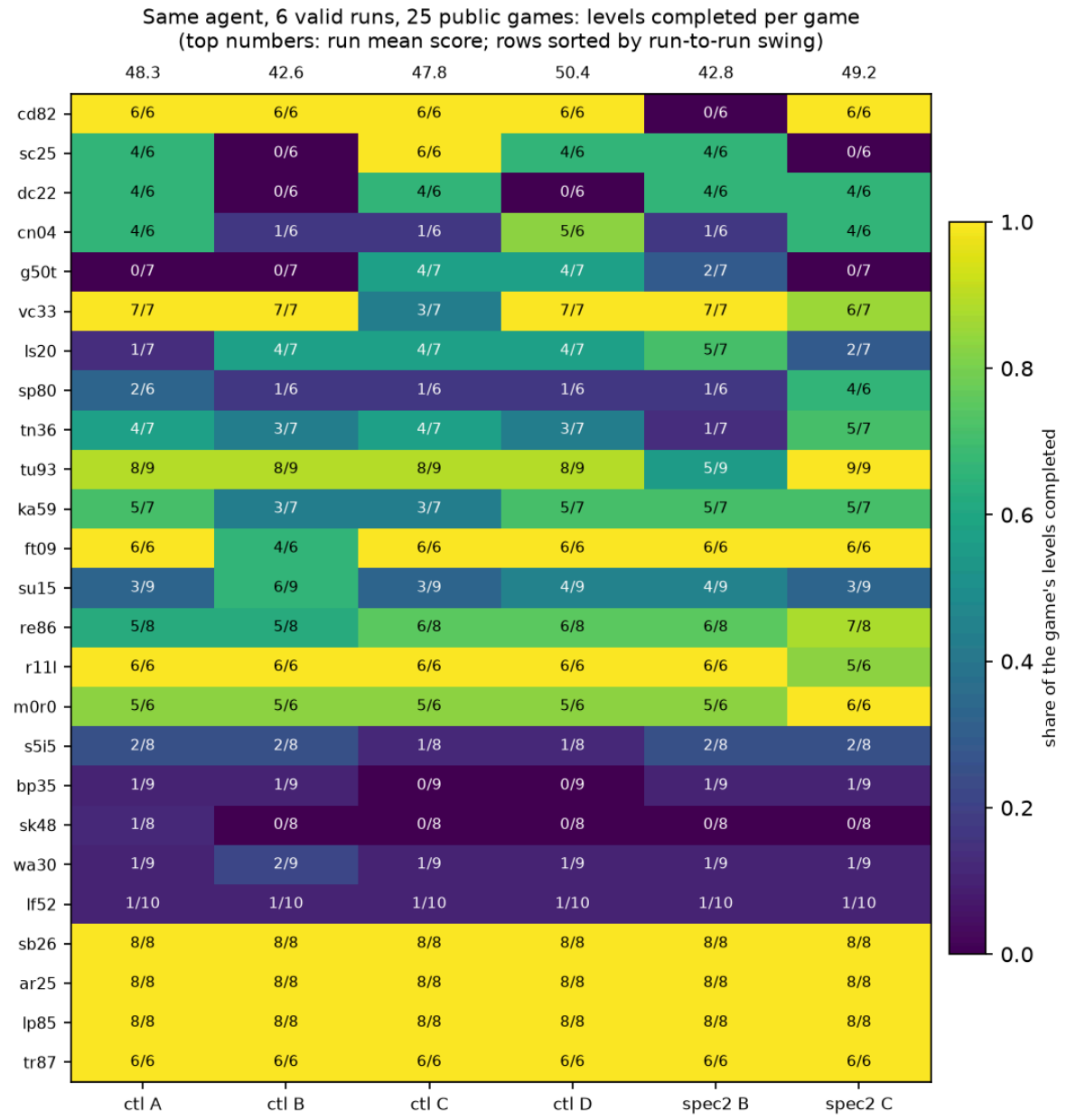


Fig. 2. Survival of level attempts in generated-token time: observed (Kaplan-Meier) vs the fitted crack model.

4.3 The noise is luck near the budget edge

Six FB25 runs of one agent (four controls and two with a serving-only change) cleared 95 to 107 levels (mean 102.5, SD 5.3). Three findings locate this noise.

It is per game (Fig. 3). Across the 13 valid runs of the B12 family, the per-game score variances, summed over games, predict an SD of the run mean of 3.79 against 3.23 observed. Per-game luck therefore accounts for 138% of the run-level variance (bootstrap 90% interval 88% to 260%): all of it, with games slightly negatively coupled. Per-game scores are uncorrelated within a run (mean pairwise correlation -0.007), and across the 13 valid runs of this agent family 22 of the 25 games change their level count between runs, by up to 7 levels (Fig. 3 shows six of these runs).



Source: benchmark.json of 6 dev runs (B12 harness; spec2 = speculative depth 2, judged noise). spec2 A left out: 24 of 25 games played.

Fig. 3. Per-game level counts across the six valid runs of one agent: the lottery is per game.

It is not throughput. Total tokens per run vary little (4.31M to 4.60M, a CV of about 2.5%) and do not track levels (correlation -0.24). We re-read each run's logs at a fixed token budget per game instead of the fixed clock. That halves the per-game variance early in the run (ratio 0.47 at a quarter of the budget) but not at the end (0.97 at the full budget; Fig. 4). At half the budget, run totals are steadier than independent games would make them (permutation test, $p = 0.021$, but one of four budget points tested, so it does not survive a Bonferroni correction), as expected when games share one server and trade tokens.

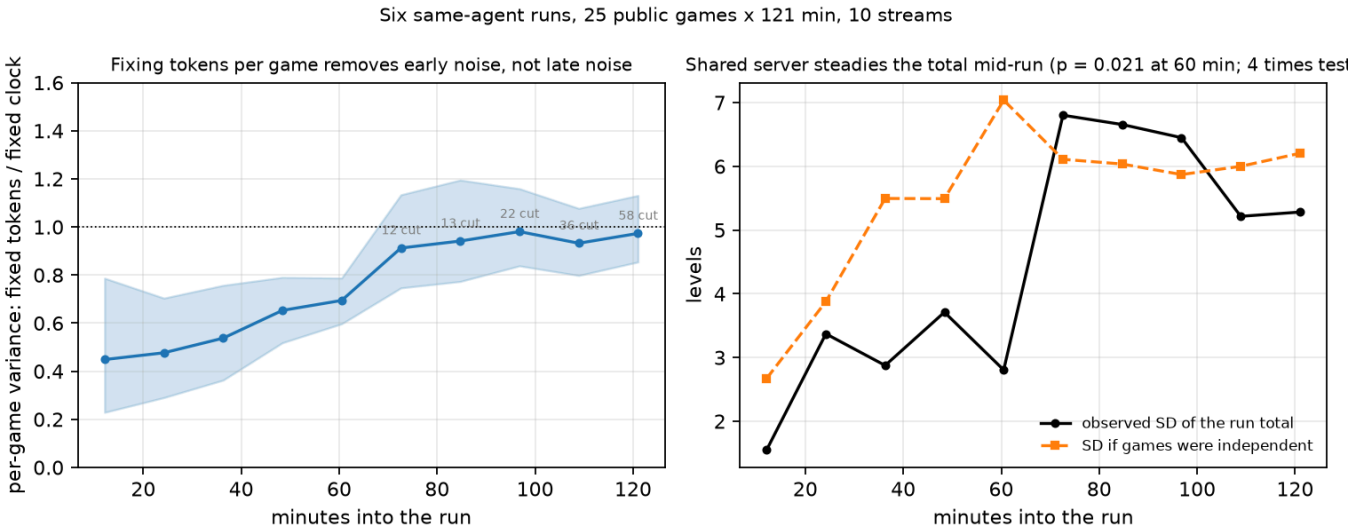


Fig. 4. Variance of per-game levels over the run: fixed clock vs fixed tokens per game.

It sits on the hard levels, and they stay hard. A model that redraws every level's difficulty and walls in every run (H0) gives a run-to-run SD of 12.3 levels. Fixing each level's difficulty but redrawing walls (H1) gives 11.1. Fixing both (H2) gives 5.6 (10-90% range 3.9 to 7.4) against 5.3 observed. At the end of a run about 17 attempts are still open (69 across the four controls); the four-run fit calls 42% of them walls and the rest slow cracks.

The rest of this section is exploratory, written after the doubled run (V5, Section 4.4) and the token-supply runs.

Fewer walls than the four-run fit says. Refitted on all eight runs of this agent (1,003 attempts, including V5), the wall share falls from 6% to 4% of levels, and the refit calls 28% of the controls' open attempts walls. It has seen V5, so this is a consistency check, not a test: it expects 101.2 levels at 121 minutes and 140.2 at twice the budget (observed: 102.75 on average and 141). Fourteen levels were reached but never cleared in any of the eight runs; the refit makes 9 of them walls with probability 0.5 or more, the clearest being level 5 of dc22, left open after 880k tokens in total. If those nine are fixed walls, unlimited play caps this agent at 154 of the 183 public levels (Appendix E).

The walls are in the games we did not develop on. Our early development runs played 10 of the 25 public games (Fig. 1, right); the other 15 were a holdout set, played only to check candidate changes (Appendix B), until the 121-minute runs used all 25. Fitted separately on the four controls, the 10 dev games show no walls (q 0.000, median wait 16.9k tokens, s 1.00; 90% of their levels cleared) and the 15 held-out games all of them (q 0.110, median 35.7k, s 1.05; 34% cleared). On all eight runs the held-out games give q 0.070 and a median of 40.3k. The pooled three numbers therefore average two different populations, and the held-out games, which we did not tune on, are probably the better guide to the hidden leaderboard games. Section 5 gives the unlimited-play caps for both game sets.

A first look inside the walls. An exploratory check that uses only the action logs (Appendix E) compares the actions the agent took on each of the 9 likely walls with the human baseline h that the scorer uses. On the held-out levels it clears, the agent needs a median of 0.72 h actions. In 6 of the 9 walls, some run took at least that many and still found no way through, so these look like failures to find the goal or the mechanic, not a shortage of moves. In the other 3, no run came close: dc22 level 5 asks people for 324 actions, and at the agent's 1,950 tokens per action there, reaching 0.72 h would take about 456k tokens, roughly twice the 218k it got in the doubled run.

Half of the per-game noise is the first level. Levels are played in order, so a game that never clears level 1 scores nothing. Over the seven 121-minute runs, the split between 0 levels and at least one accounts for 49% of the summed per-game variance (bootstrap over runs, 90% interval 39% to 57%), almost all of it in four games (sc25, cd82, dc22 and g50t). These zeros are not walls: 14 of the 17 game runs that ended with 0 levels were shelved by the harness's slot rule, which at each context trim puts a game behind games of higher priority, and a game's priority falls with the tokens and actions spent on its current level (Appendix C), and in V5 all 6 of these games cleared level 1. A game's first crack therefore races the slot rule's patience as well as the clock (Appendix E). A sealed one-run test that protects games on level 1 from shelving passed (V11, Section 4.4): 1 of the 23 games that started ended with 0 levels, but 2 games never started.

4.4 Sealed predictions

The five sealed tests at a glance (times UTC; one paragraph per test below, and every rule and readout in full in Appendix D):

Test	Sealed	Public time stamp	Outcome
Private ARC-AGI-3 board, lottery model H against low noise S	2 Nov	hash posted on Kaggle (discussion 755123); Bitcoin block 974295	pending
V5: twice the play time	4 Nov, pushed to GitHub at 10:18; run started 10:28	Bitcoin block 974519 at 10:52, 24 minutes into the run	pass: 141 levels, inside 110 to 145 but near the top
V10: less token supply	4 Nov, committed at 16:42	Bitcoin block 974554	unscored: both runs left games unplayed
V11: first-level guard	5 Nov, committed at 14:32; kernel pushed 14:40	Bitcoin block 974681 at 15:34	pass in one partial run: 1 of 23 started games at 0 levels; 2 never started
Private ARC-AGI-2 board (V9)	4 Nov	hash in a comment on the same Kaggle post; Bitcoin block 974554 at 17:27	pending

Until its Bitcoin anchor, a sealed file is dated only by the history of our working repository on GitHub. That repository is private, and the public one is a later one-commit copy, so readers cannot check those times. For V5 this matters most, because its anchor came 24 minutes into the run. The Bitcoin proofs are in analysis/s37/sealed, and anyone can check them with OpenTimestamps.

Private ARC-AGI-3 board (sealed 2 November; pending). Before Kaggle published the private board, we wrote down what the lottery model (H) and a low-noise alternative (S) predict for the crowd of teams with final public scores from 20 to just under 40 (2,259 teams when we fitted the model). The two disagree sharply: for example, the rank correlation of public and private scores within the crowd is 0.39 under H and 0.90 under S, and H fails above 0.70. Four such tests and a decision rule are fixed in the file; score_sealed.py applies the rule when the hosts publish the board.

Twice the play time (V5, sealed 4 November; pass). The crack model predicted that doubling the play time per game, everything else byte-identical, would raise the level total from 102.75 to 127 (90% interval 110 to 145). The run cleared 141 levels (Fig. 5), and the three secondary checks also landed inside their ranges. The pass is not comfortable: 141 is 14 above the point prediction and five below the "out of time" line. A later simulation of the harness's slot rule traces about half of the gap to an endgame rule that shaped the control runs the sealed calculation continued (Appendix D).

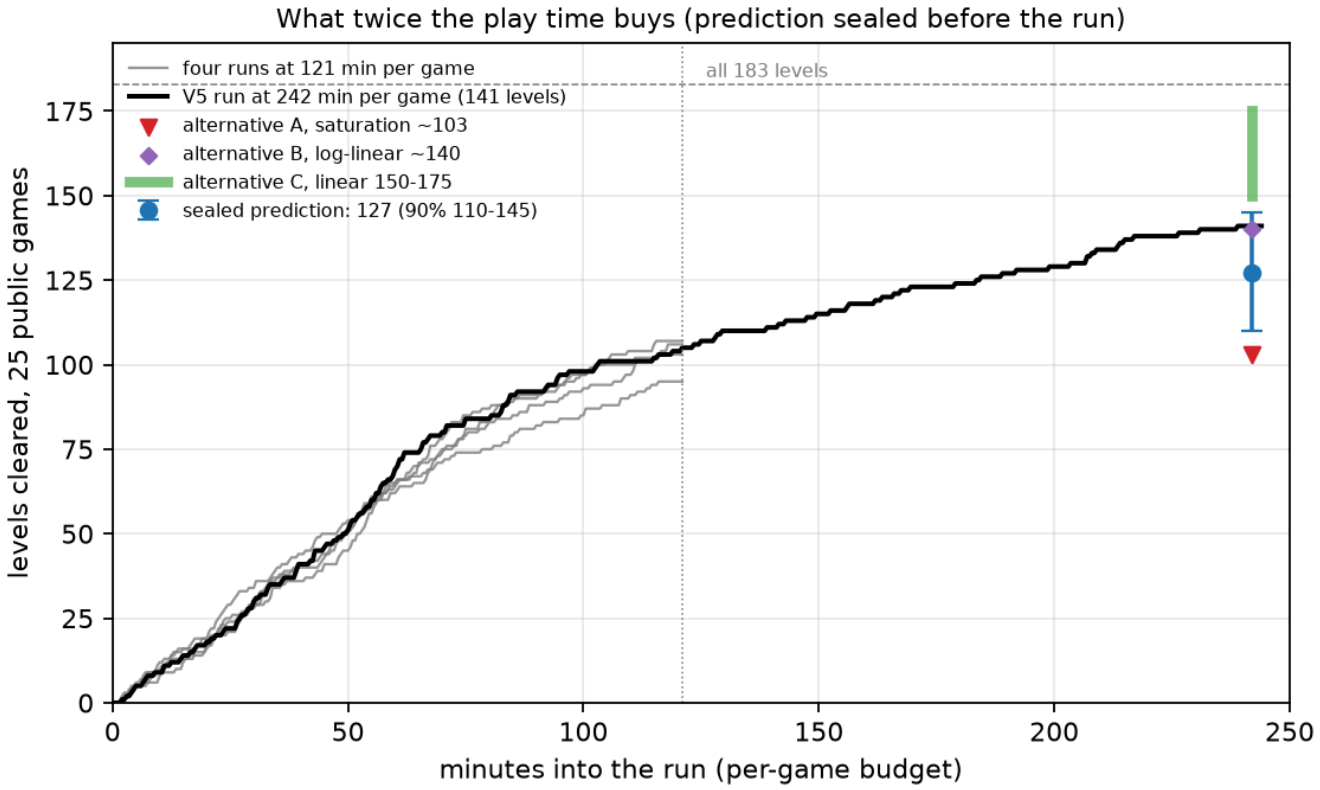


Fig. 5. Levels vs play time: four runs at 121 min per game, the V5 run at twice the play time, the sealed prediction (127, 90% 110-145) and three alternatives.

Less token supply (V10, sealed 4 November; unscored). With speculative decoding off, a run that generates fewer tokens on the same clock should play like a control run stopped early. Both runs broke the file's validity rule, because games were left that never started (2 in the first run, 1 in the rerun), so the prediction is untested. Exploratory only: their totals, 86 and 82 levels, lie inside the sealed bands at their measured supply.

A first-level guard (V11, sealed 5 November; pass in one partial run). A game on level 1 gets top slot priority until it clears the level or has spent 150k tokens on it. The sealed rule passes the guard if at most 1 of the games that start ends with 0 levels; the seven reference runs had 1 to 4, so with no effect a pass has about a 1 in 7 chance. V11 had 1 of the 23 games that started, but 2 games never started; counting them as zeros gives 3.

Private ARC-AGI-2 board (V9, sealed 4 November; pending). The lottery model, refitted to the plateau of Section 4.8, predicts how its 2,133 teams reorder on the private board: for example, a rank correlation of public and private scores of 0.82 (0.65 to 0.91). Three named alternatives carry their own values, and score_v9.py applies the rule when the board is published.

Appendix D gives each file's full predictions, decision rule, time stamps and readout, with the analyses made after the V5 and V11 results.

4.5 Our 32 negative results, sorted by what each change did

We sorted the negative results after the fact by what each change did to the run. The sorting describes them; it does not test the model.

- Fewer tokens for the levels that count. Rolling memory summaries cut prefix-cache hits from 94% to 69% and agent turns by 29%, so the same budget bought fewer attempts. ThinkingCap, a variant of our language model tuned to think less, generated 3.52M tokens in the run instead of about 4.4M.
- Higher crack cost. Low reasoning effort halved the level count.
- No change to cracking. More concurrent streams and shallower speculative decoding (one or two draft steps instead of three; both in Appendix A) changed serving, not what the agent does; a scene-digest tool was never

called (0 of 541 calls). On the 14 valid FB25 runs with token logs, crack costs b range from -0.150 to +0.060, against an A/A SD of 0.051. None of the 6 variants with valid runs passes a multiple-comparison line (Bonferroni over the 6 variants, |z| of 2.64 or more); ThinkingCap's cheaper cracks come closest and reach the line only after rounding (one run). Next is the ht harness (mean b -0.095 over two runs, z 2.2), the one candidate worth a third run; two-step speculative decoding (spec2, mean b -0.048 over its two valid runs) is 1.1 standard errors from the control. An A/A check made after the outside review asks how firm that line is. Left out in turn and tested like a variant (analysis/s37/aa_calibration.py), five of the six same-agent runs sit within 0.4 standard errors of the control, but b32_spec2_r3 sets most of the A/A SD: without it the other five spread only 0.015, and 5 of the 6 variants would cross the line. A level bootstrap of the same fit (400 refits, analysis/s37/aa_bootstrap_b.py) sides with the six-run SD: it puts the standard error of a one-run variant's b at 0.057 to 0.072, b32_spec2_r3 at z -1.97, and none of the 6 variants past the line (largest |z| 2.09 for ThinkingCap, then 2.01 for ht). The tight cluster of the other five runs is the unusual part, so 'none passes' stands, but with six A/A runs the line is soft. Two runs do not fit the two numbers. ThinkingCap's cost per level fell (b -0.150) but it cleared 13.1 levels fewer than predicted, which in the model means more walls. B15C generated 23% more tokens (5.45M) and still cleared 6.9 levels fewer than predicted: more supply did not turn into more cracks.

4.6 How many runs it takes to see a real change

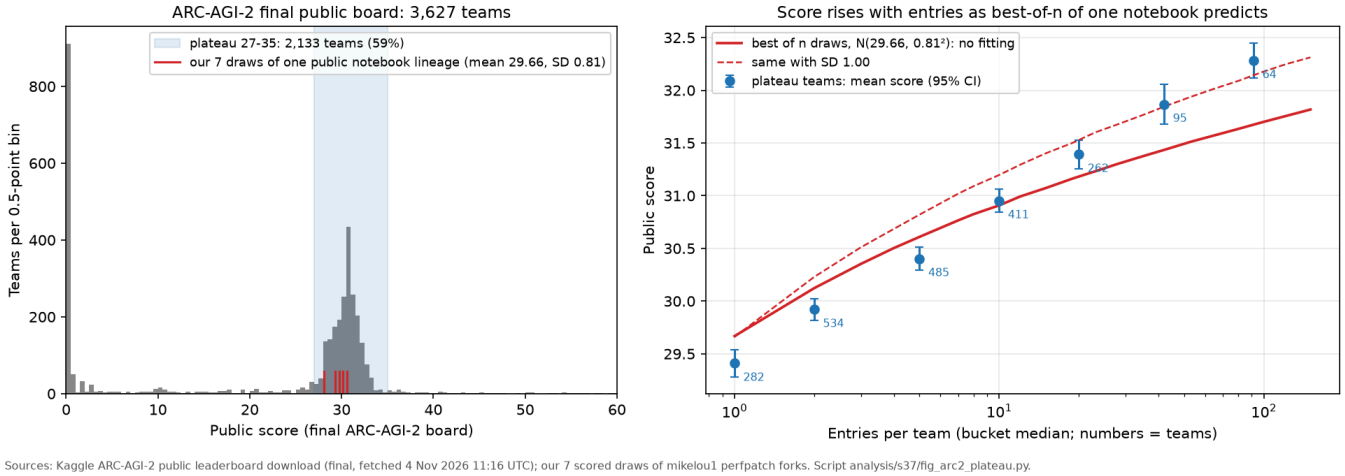
On the leaderboard one submission has an SD of 2.87 points. To detect a true difference with power 0.8 (two-sided test at 0.05) takes 15 submissions per arm for a gain of 3 points and 6 for a gain of 5; one run per arm resolves only 11.4 points. The SD comes from five submissions, so the 15 could be anywhere from 7 to 81. Resubmission alone buys points: the best of 10 identical submissions beats their mean by 4.42. In development runs, crack cost b needs several times fewer runs than raw level totals: 4.5 against 22.6 per arm on the 6 valid same-agent runs, and at least about 2.8 times fewer when any one run is left out. A CPU notebook (paper/notebook/how_many_runs.ipynb in our repository, also public on Kaggle at kaggle.com/code/shreekumarshah/how-many-runs-do-you-need-arc-agi-3) computes these run counts and fits b for two arms from a team's own run logs. Appendix E gives the t-test counts and the full best-of-n series.

4.7 Methods the model rules out

We simulated schedulers under the fitted model before spending GPU time on them. Giving the 10 streams to the games with the highest current crack hazard gains +0.0% over equal sharing when games are alike and -3.6% when their difficulty differs, while an oracle that knows which games are walled and skips them gains +16.9% and +13.4%. A scheduler is therefore worth building only with a wall detector, and a detector trained on run logs was close to chance (leave-one-game-out AUC 0.491 to 0.582; Appendix E), so we did not build it.

4.8 The same lottery elsewhere

ARC-AGI-2. On the final public ARC-AGI-2 board (3,627 teams), 2,133 teams (59%) sit on a plateau between 27 and 35. If these teams mostly resubmit near-identical public notebooks, each team's score is roughly the best of its n draws. Without fitting anything to the board, the best of n draws from seven scored draws of one public notebook lineage (mean 29.66, SD 0.81; Table 2) predicts a plateau mean of 29.65 for single-entry teams rising to 31.67 at 91 entries; observed means rise from 29.41 (282 teams with one entry) to 32.28 (64 teams with 64 or more) (Fig. 6). From 1 to 91 entries that is 2.01 of the observed 2.87 points (70%). Run luck alone, at the pooled SD of our unchanged resubmissions (0.61), gives 1.50 (52%); the rest mixes small code differences with teams improving their code between entries. Run luck is thus about half of the effect, with wide bounds from only seven draws (Appendix E). Section 4.4 gives a sealed prediction of how this plateau reorders on the private board.



Sources: Kaggle ARC-AGI-2 public leaderboard download (final, fetched 4 Nov 2026 11:16 UTC); our 7 scored draws of mikelou1 perfpatch forks. Script analysis/s37/fig_arc2_plateau.py.

Fig. 6. ARC-AGI-2: the plateau and its rise with entries, against best-of-n draws of one notebook.

Other teams' harnesses. Fitted separately to six public runs by other teams, the crack model reproduces each run's level total within 1.3 levels. Four forks of the Duck harness show our median crack wait (20.2k to 28.1k tokens) but more walls (q from 0.22 to 0.37); two runs show no walls (q = 0.00).

Another benchmark (exploratory, added 7 Nov). Bjarnason et al. ran each of three models in two scaffolds, at two temperatures, ten times on SWE-Bench-Verified and published the runs [Bjarnason26]. We applied the check of Section 4.3 to all 12 configurations (120 runs over 500 tasks): independent per-task luck, computed from each task's success rate across a configuration's runs, predicts 86% of the observed run-to-run variance of the resolution rate when pooled, inside the 81% to 127% that pure per-task luck would give (near its low end), and 51% to 200% per configuration. Between 144 and 309 of the 500 tasks are solved in some runs and not in others. On this coding benchmark too, the run noise is per-task luck; whether it also sits near a budget edge would need a fit to their token logs (Appendix E).

One caveat on independence. dfranzen's public notebook plays four passes over the 25 games, and their level totals (102, 106, 104 and 105) vary far less than independent games would make them (p = 0.020), which fewer level-1 zeros do not explain (Appendix E). Some harnesses therefore couple passes, and for them our independent-game noise is an upper bound. We have not found the mechanism.

5. Discussion: what this means for progress

What can raise a score. In the model, a change raises the score only if it raises cracks per GPU-second, and one run shows the gain only if it is larger than the noise floor. Two routes qualify. One is more tokens for the games that need them: faster serving, fewer wasted tokens, fewer games stalled at the end, and more patience for slow first levels (about half of the per-game noise is whether a game clears level 1 before the harness's slot rule shelves it, Section 4.3; a guard against that shelving passed a sealed one-run test, but 2 games never started, Section 4.4). Frontier results point the same way: keeping the model's reasoning and compacting the history tripled GPT-5.6 Sol's ARC-AGI-3 public-set score (13.3% to 38.3%) with 6 times fewer output tokens [OpenAI26]. Doubling the play time bought 38 more levels (141 against 102.75 at 1x), more than the 24 the sealed calculation predicted; in simulation the harness's endgame barely changes what doubling buys, but it shaped the control runs that the calculation continued, which explains about half of the gap (Section 4.4 and Appendix D). The other is fewer walls. At the end of a control run 28% to 42% of the open attempts are walls in the model (refitted with the doubled run, and fitted on the four controls; Section 4.3). A first look at the action logs splits them in two (Section 4.3): on 6 of the 9 likely walls the agent took as many moves as it usually needs and still missed the goal, while on long levels such as dc22 level 5 it thinks too long per move to get there, so cheaper moves are a route too. A scheduler that knew which games were walled would gain 13% to 17% in simulation, while one that only chases the highest crack hazard gains nothing (and loses 3.6% when the games differ in difficulty), and our attempt to predict walls from the logs scored close to chance (AUCs 0.49 to 0.58; Section 4.7). Changes that only reshape how the agent talks or remembers, without changing either route, should be expected to fail, and ours did.

How to tell a real gain from luck. One leaderboard draw per arm resolves only differences of 11.4 points or more. A team's best of 100 identical entries sits about 7 points above its mean. Inside a crowded band, rank therefore depends

heavily on the number of entries and on luck, not only on the code. For agent comparisons in development, comparing crack cost on shared levels needed several times fewer runs than comparing level totals (5.1 times on our six valid runs, at least 2.8). We suggest that papers and leaderboards report the SD and the number of runs, and that leaderboards show the number of entries next to each best score.

Why this matters for the 85% goal. A field that reads leaderboards learns from differences. If many of those differences between similar agents are luck, the field learns slowly. A model that predicts the size of the luck tells a team when a result is worth acting on, and which kinds of change could ever beat it. In the model's terms the distance is set by walls, not by time. In ARC's vocabulary, run luck sits on the compute side and walls on the skill side: extra tokens buy the cracks near the budget edge, but a wall does not crack within any budget we can afford. In this model, then, the gap to 85% closes through skill acquisition (fewer walls), not through more play alone. Frontier results suggest what that skill is. The hosts report that once a frontier model understands a game's mechanics, it plays within the range of human action efficiency [ARCAstra26], and in our logs most walls are levels where the agent made its usual number of moves and still missed the goal (Section 4.3): failures to understand rather than to execute. A harness that keeps the model's reasoning state between requests lifted one frontier model from 62.7% to 99.9% [ARCAstra26]; our X6 ledger, which keeps each solved level's rule in the prompt, is a small open-model version of the same idea. At the wall rate fitted on the four controls (6% of levels), even unlimited play caps this agent at about 73% on the 25 public games, and at the 4% refitted with the doubled run, at about 81%. On the 15 held-out games, the better guide to the hidden games (Section 4.3), the caps are lower: about 56% at their four-control wall rate (q 0.110) and 69% at their eight-run rate (q 0.070). On either game set, 85% needs a wall rate of 3.1% or less, and without walls it would take about 1.6 (four-run fit) to 1.9 (refit) times the current budget. The 100% line of the ARC-AGI-3 grand prize needs no walls and move counts that match humans on aggregate (matching them on every level is enough but not required; an extrapolation of the fitted model, not a measurement).

6. Limitations

- One agent family (the Duck harness through dfranzen's notebook and our B12), one language model, one GPU type.
- Development runs use the 25 public games. The hidden games are harder (B12's four control runs average 47.3 on the public games in development, against 28.85 on average on the leaderboard), so the fitted numbers describe the public games. The 10 games we developed on show no walls and the 15 we held out show all of them (Section 4.3), so the pooled fit flatters the agent on new games.
- The harness schedules games differently over the last 40% of each game's budget, so the end state of a run is not the state of a longer run at the same minute. The sealed V5 calculation continued such end states; the simulator that sizes this (Appendix D) is fitted to the same runs and misses one of our checks.
- The model treats walls as permanent. In the doubled run, attempts past 150k tokens cracked twice as often as the fitted hazard expects (6 against 2.9), so some walls may be very slow cracks. Refitted with that run, the wall share falls from 6% to 4% (Section 4.3).
- Most analyses are exploratory: they were written after seeing data. Only the V5, V10 and V11 predictions and the private-leaderboard predictions were sealed before their data existed; V10 went unscored, and V11 is one partial run (Section 4.4).
- Games wait for a turn: the harness plays at most 10 games at a time and counts every game's budget from the start of the run, so the last games to start get less time (in the four controls and T1c the last one started 69 to 99 minutes into its 121, and in the slower T1 runs and in V11 some never started). Start order follows the number of levels, so the longest games start last (Appendix C).
- Some harnesses couple their passes (Section 4.8), so our noise estimates may not transfer to them.
- The private leaderboard is still withheld, so the sealed private-board predictions are not yet scored.

7. Conclusion

Identical code on the ARC-AGI-3 leaderboard spans 1,723 places because each level waits for a crack whose timing is luck, and the luck falls on the hard levels near the edge of the budget; for a game stuck on its first level that edge is the harness's slot rule, which can shelve it early in the run (a guard against this passed a sealed test in one partial run). A three-number model fitted on four runs explains the size of that luck, our negative results, and the plateau of the ARC-AGI-2 board; on 120 public SWE-Bench-Verified runs, the same per-task luck accounts for most of the run noise (exploratory). A prediction sealed before a run at twice the play time held: 141 levels against 127 (90% interval 110 to 145), near the top; in simulation about half of the gap traces to a harness endgame rule that shaped the runs the sealed calculation continued. The model also says what to do: buy tokens for the games that need them, attack walls, and count runs before believing a gain.

Table 1. Our ARC-AGI-3 submissions

#	Date (UTC)	Ref	What it was	Public score	Place on final board
1	2026-09-30 15:55	56712371	B0: fork of a public notebook (scottlegrand, 5.19); pipeline check	2.65	2,529
2	2026-10-01 06:47	56745504	B1: fork of dfranzen's Milestone 2 notebook	28.30	1,486
3	2026-10-02 08:35	56769866	B1 resubmitted (noise sample 2)	26.72	1,840
4	2026-10-03 05:32	56790419	X6: B1 + ledger of solved levels	29.29	1,216
5	2026-10-04 19:50	56832595	B12: X6 ledger v2 + D' slot priority	33.37	197
6	2026-10-05 18:14	56858951	B12 resubmitted (noise sample 2)	28.24	1,503
7	2026-10-06 20:53	56890682	B15: B12 on a vLLM runtime	24.27	2,147
8	2026-10-07 19:17	56919974	B12 resubmitted (noise sample 3)	26.76	1,836
9	2026-10-08 05:15	56936216	LEN: B12 + dead-trace stub and cap notice	25.02	2,070
10	2026-10-09 09:27	57003717	A1: fork of a different public notebook (arena p50)	11.05	2,320
11	2026-10-10 20:47	57052442	B12 resubmitted (noise sample 4)	29.69	1,096
12	2026-10-11 18:19	57082387	HT2: B12 + stall detector + fresh-eyes analyst calls (second build)	26.92	1,811
13	2026-10-12 19:56	57117599	HT: B12 + stall detector + fresh-eyes analyst calls (first build)	29.88	1,040
14	2026-10-13 18:31	57153031	B12 resubmitted (noise sample 5)	26.21	1,920
15	2026-10-29 05:44	57760408	B28: B12 + 40-minute start-up timeout	25.76	1,992
16	2026-10-30 19:58	57815956	B30: B12 + 50-minute start-up timeout + end cap	23.82	2,171
17	2026-11-02 10:16	57904835	B30 resubmitted	25.54	2,017

Final public board: 5,073 teams (fetched 4 Nov 2026 09:51 UTC). Private scores: not yet published. Ref is the Kaggle submission ID.

Table 2. Our ARC-AGI-2 submissions

#	Date (UTC)	Ref	What it was	Public score	Place on final board
1	2026-09-30 15:55	56712373	A0: fork of a public notebook (mikelou1's perfpatch, public score 33.89)	29.72	1,766
2	2026-10-01 06:40	56745390	A1: A0 + dynamic token cap and a fixed hash seed	29.44	1,791
3	2026-10-02 08:35	56769864	A2: A1 + a second-pass seed ensemble on idle GPU time	28.33	2,154
4	2026-10-03 15:13	56800905	E2.1: our model-first pipeline (27B model, 48k token cap, 2 attempts)	15.69	2,460
5	2026-10-04 21:03	56833698	A0 resubmitted (noise sample 2)	29.31	1,901
6	2026-10-05 18:14	56858952	A0 resubmitted (noise sample 3)	28.06	2,221
7	2026-10-09 17:59	57016836	PP: the public minimal-perfpatch notebook, unchanged	30.14	1,571
8	2026-10-11 18:23	57082458	PP resubmitted (noise sample 2)	29.72	1,766
9	2026-10-12 19:57	57117601	PP resubmitted (noise sample 3)	30.14	1,571
10	2026-10-13 18:31	57153034	PP resubmitted (noise sample 4)	30.56	1,322

Final public board: 3,627 teams (fetched 4 Nov 2026 11:16 UTC). Private scores: not yet published. Ref is the Kaggle submission ID. The seven A0 and PP draws are the "one public notebook lineage" of Section 4.8.

References

- [Agarwal21] R. Agarwal et al. "Deep Reinforcement Learning at the Edge of the Statistical Precipice." NeurIPS 2021. arXiv 2108.13264.
- [Akyurek24] E. Akyürek, M. Damani, A. Zweiger, L. Qiu, H. Guo, J. Pari, Y. Kim, J. Andreas. "The Surprising Effectiveness of Test-Time Training for Few-Shot Learning." arXiv 2411.07279 (2024; title as of v2).
- [ARC2] F. Chollet, M. Knoop, G. Kamradt, B. Landers, H. Pinkard. "ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems." arXiv 2505.11831.
- [ARC24] F. Chollet, M. Knoop, G. Kamradt, B. Landers. "ARC Prize 2024: Technical Report." arXiv 2412.04604.
- [ARC3] ARC Prize Foundation. "ARC-AGI-3 Technical Report" (arcprize.org/media/ARC_AGI_3_Technical_Report.pdf). Section 3.5.1: games qualified so random play cannot beat non-tutorial levels (1M-step random sweeps; P_win < 1/10,000 per level); scoring compares AI to an upper-median human baseline per level, then aggregates per game.
- [ARC3Kaggle] Kaggle. "ARC Prize 2026 - ARC-AGI-3", Prizes (kaggle.com/competitions/arc-prize-2026-arc-agi-3/overview/prizes), read 4 Nov 2026: the Grand Prize is unlocked if a team achieves a score of 100% on the competition leaderboard.
- [ARCAstra26] ARC Prize Foundation. "OpenAI's GPT-6 Astra on ARC-AGI-3." ARC Prize blog, 2026 (arcprize.org/blog/astra; read 6 Nov 2026: 62.7% on the semi-private set with the Standard harness, 99.9% with the Provider Adapter harness; once frontier AI understands the mechanics it generally executes within the range of human efficiency, a binary-like pattern).
- [ARCo3] F. Chollet. "OpenAI o3 Breakthrough High Score on ARC-AGI-Pub." ARC Prize blog, 20 Dec 2024 (arcprize.org/blog/oai-o3-pub-breakthrough; read 6 Nov 2026: 75.7% on the semi-private set within the $10k compute limit, 87.5% at 172 times the compute).
- [ARCPaper26] ARC Prize Foundation. "ARC Prize 2026 Paper Prize" (arcprize.org/competitions/2026/paper), read 4 Nov 2026: Progress asks "how much does the paper increase the overall chance of anyone achieving 85% on ARC-AGI?"
- [BerksonGage52] J. Berkson, R. P. Gage. "Survival curve for cancer patients following treatment." Journal of the American Statistical Association 47(259): 501-515, 1952. doi:10.1080/01621459.1952.10501187
- [Bjarnason26] B. H. Bjarnason, A. Silva, M. Monperrus. "On Randomness in Agentic Evals." arXiv 2602.07150 (2026; abstract checked 6 Nov 2026: 60,000 SWE-Bench-Verified trajectories; single-run pass@1 varies by 2.2 to 6.0 points with the run chosen).
- [BlumHardt15] A. Blum, M. Hardt. "The Ladder: A Reliable Leaderboard for Machine Learning Competitions." ICML 2015, PMLR 37.
- [Bouthillier21] X. Bouthillier et al. "Accounting for Variance in Machine Learning Benchmarks." MLSys 2021. arXiv 2103.03098 (abstract checked 4 Nov 2026: variance from data sampling, initialization and hyperparameter choice).
- [Brown24] B. Brown et al. "Large Language Monkeys: Scaling Inference Compute with Repeated Sampling." arXiv 2407.21787.
- [Chollet19] F. Chollet. "On the Measure of Intelligence." arXiv 1911.01547.
- [Colas18] C. Colas, O. Sigaud, P.-Y. Oudeyer. "How Many Random Seeds? Statistical Power Analysis in Deep RL Experiments." arXiv 1806.08295.
- [Dodge19] J. Dodge et al. "Show Your Work: Improved Reporting of Experimental Results." EMNLP-IJCNLP 2019, pp. 2185-2194 (aclanthology.org/D19-1224). Expected best validation score as a function of the number of trials.
- [Dwork15] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, A. Roth. "The reusable holdout: Preserving validity in adaptive data analysis." Science 349(6248), 2015. doi 10.1126/science.aaa9375 (checked 6 Nov 2026; STOC 2015 version arXiv 1411.2664).

- [Farewell82] V. T. Farewell. "The Use of Mixture Models for the Analysis of Survival Data with Long-Term Survivors." Biometrics 38(4):1041-1046, 1982. doi 10.2307/2529885. (Our "wall" share = the cured / never-event fraction.)
- [Franzen25] D. Franzen, J. Disselhoff, D. Hartmann. "Product of Experts with LLMs: Boosting Performance on ARC Is a Matter of Perspective." arXiv 2505.07859.
- [Gomes00] C. P. Gomes, B. Selman, N. Crato, H. Kautz. "Heavy-Tailed Phenomena in Satisfiability and Constraint Satisfaction Problems." Journal of Automated Reasoning 24:67-100, 2000. doi 10.1023/A:1006314320276 (random restarts eliminate heavy tails).
- [Greenblatt24] R. Greenblatt. "Getting 50% (SoTA) on ARC-AGI with GPT-4o." Redwood Research blog, 17 June 2024 (blog.redwoodresearch.org/p/getting-50-sota-on-arc-agi-with-gpt; read 5 Nov 2026: about 8,000 sampled programs per task, "an additional 3% correct per doubling of k").
- [Hardy26] M. Hardy, R. Azam, A. Reuel, M. Kochenderfer, S. Koyejo. "Agent Evaluation Reliability: More Tasks Won't (Always) Fix An Agent Leaderboard." arXiv 2610.00651 (2026; abstract checked 6 Nov 2026: Bayesian variance decomposition, 22 agent benchmarks).
- [Heineman25] D. Heineman, V. Hofmann, I. Magnusson, Y. Gu, N. A. Smith, H. Hajishirzi, K. Lo, J. Dodge. "Signal and Noise: A Framework for Reducing Uncertainty in Language Model Evaluation." arXiv 2508.13144 (Aug 2025).
- [Henderson18] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger. "Deep Reinforcement Learning That Matters." AAAI 2018 (vol. 32). arXiv 1709.06560. (ojs.aaai.org/index.php/aaai/article/view/11694)
- [HoosStutzle98] H. H. Hoos, T. Stützle. "Evaluating Las Vegas Algorithms - Pitfalls and Remedies." UAI 1998. arXiv 1301.7383 (abstract checked 5 Nov 2026: evaluation by empirical run-time distributions).
- [KapoorAgents24] S. Kapoor et al. "AI Agents That Matter." arXiv 2407.01502.
- [Kwa25] T. Kwa, B. West, J. Becker et al. "Measuring AI Ability to Complete Long Software Tasks." arXiv 2503.14499 (2025; NeurIPS 2025; checked 6 Nov 2026: the 50% time horizon, in human working time, of tasks an agent completes).
- [Li24] W.-D. Li, K. Hu, C. Larsen, Y. Wu, S. Alford, C. Woo, S. M. Dunn, H. Tang, M. Naim, D. Nguyen, W.-L. Zheng, Z. Tavares, Y. Pu, K. Ellis. "Combining Induction and Transduction for Abstract Reasoning." arXiv 2411.02272 (2024).
- [Luby93] M. Luby, A. Sinclair, D. Zuckerman. "Optimal speedup of Las Vegas algorithms." Information Processing Letters 47(4):173-180, 1993. doi 10.1016/0020-0190(93)90029-9.
- [Madaan24] L. Madaan et al. "Quantifying Variance in Evaluation Benchmarks." arXiv 2406.10229.
- [MallerZhou96] R. Maller, X. Zhou. Survival Analysis with Long-Term Survivors. Wiley, Chichester, 1996 (271 pp.).
- [Miller24] E. Miller. "Adding Error Bars to Evals: A Statistical Approach to Language Model Evaluations." arXiv 2411.00640 (Nov 2024; checked 5 Nov 2026: recommends clustered standard errors and question-level paired differences).
- [Nosek18] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor. "The preregistration revolution." PNAS 115(11):2600-2606, 2018. doi 10.1073/pnas.1708274114.
- [OpenAI26] OpenAI. "How enabling two settings tripled our scores on the ARC-AGI-3 benchmark." 29 July 2026 (openai.com/index/how-two-settings-tripled-our-arc-agi-3-scores; read 6 Nov 2026: GPT-5.6 Sol 13.3% to 38.3% on the public set with retained reasoning and compaction, 6x fewer output tokens).
- [Polo24] F. Maia Polo, L. Weber, L. Choshen, Y. Sun, G. Xu, M. Yurochkin. "tinyBenchmarks: evaluating LLMs with fewer examples." arXiv 2402.14992 (ICML 2024).
- [PreregML20] L. Bertinetto, J. F. Henriques, S. Albanie, M. Paganini, G. Varol (eds.). NeurIPS 2020 Workshop on Pre-registration in Machine Learning. PMLR 148, 2021 (proceedings.mlr.press/v148, checked 5 Nov 2026).
- [Roelofs19] R. Roelofs et al. "A Meta-Analysis of Overfitting in Machine Learning." NeurIPS 2019. Over 100 Kaggle competitions, public vs final ranking: little evidence of substantial overfitting (abstract checked 4 Nov 2026, papers.nips.cc paper 9117).

- [Schaeffer25] R. Schaeffer, J. Kazdan, J. Hughes, J. Juravsky, S. Price, A. Lynch, E. Jones, R. Kirk, A. Mirhoseini, S. Koyejo. "How Do Large Language Monkeys Get Their Power (Laws)?" ICML 2025 (PMLR 267). arXiv 2502.17578 (checked 5 Nov 2026: aggregate power laws from a heavy-tailed distribution of per-problem success probabilities).
- [Shinn23] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. "Reflexion: Language Agents with Verbal Reinforcement Learning." NeurIPS 2023.
- [Singh25] S. Singh, Y. Nan, A. Wang, D. D'Souza, S. Kapoor, A. Üstün, S. Koyejo, Y. Deng, S. Longpre, N. A. Smith et al. "The Leaderboard Illusion." arXiv 2504.20879 (2025; checked 5 Nov 2026: undisclosed private testing of many variants on Chatbot Arena and selective disclosure of the best scores bias the leaderboard).
- [Snell24] C. Snell, J. Lee, K. Xu, A. Kumar. "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters." arXiv 2408.03314.
- [SOAR] J. Pourcel, C. Colas, P.-Y. Oudeyer. "Self-Improving Language Models for Evolutionary Program Synthesis: A Case Study on ARC-AGI" (SOAR). openreview z4IG090qt2.
- [Stroebl24] B. Stroebl, S. Kapoor, A. Narayanan. "The Limits of Inference Scaling Through Resampling." arXiv 2411.17501 (2024; authors corrected 5 Nov 2026 from the arXiv page).
- [Wang23] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, A. Anandkumar. "Voyager: An Open-Ended Embodied Agent with Large Language Models." arXiv 2305.16291 (2023; a skill library of programs).
- [Wiedmann26] L. Wiedmann, L. Girrbach, C. Schmid, Z. Akata. "Agents Are Systems, Not Models: Rethinking Agentic Evaluation." arXiv 2610.01618 (2026; abstract checked 6 Nov 2026: about 54% of outcome variance from repeating the same configuration).
- [Yao24] S. Yao et al. "tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." arXiv 2406.12045 (pass^k).

Appendix A. The 32 negative ledger rows

#	Ledger id	Date	Track	Hypothesis (short)	Verdict
1	E1b-qwen	2026-10-01	ARC-AGI-2 eval 120	Qwen3.8-27B FP8 L4x4 K=4 16k	KILLED
2	B3v2-X2-rep	2026-10-01	ARC-AGI-3	X2 identical replicate	NOISE
3	B5-X2-HO-res	2026-10-01	ARC-AGI-3	X2 on 15 holdout games	KILLED
4	A1-LB	2026-10-02	ARC-AGI-2	A0+dyncap	NOISE
5	B1-rerun-LB	2026-10-02	ARC-AGI-3	Franzen fork rerun	NOISE
6	A2-LB	2026-10-03	ARC-AGI-2	pass2 ensemble	NOISE
7	E3T	2026-10-03	ARC-AGI-2 eval 120+60	TPU v5e-8 bf16 vllm-tpu 0.28: phase A low16k 120 tasks (9000 s) + phase B xhigh48k 60 tasks (12600 s	FAIL (infra)
8	B8-X7	2026-10-03	ARC-AGI-3	X7 playbook summary compaction on X6 (interval 16k < drain 58k; stop drain at summaries; hide follow	FAIL (killed)
9	B9-X8	2026-10-03	ARC-AGI-3	X8 prediction-mismatch halt as prompt (existing ARC3_STEP_VERIFICATION_HINT=1) on X6	FAIL gate (neutral)
10	E2.1	2026-10-03	ARC-AGI-2 eval 60	model-first 27B GPTQ xhigh 48k cap, 2 attempts, forced answer on truncation	FAIL accuracy (T2 mechanics pass)
11	B10-X9	2026-10-03	ARC-AGI-3	X9 reasoning_effort=low on every agent turn (template default xhigh); compute-bound evidence says mo	FAIL (killed)
12	R4-FS2-G0	2026-10-03	ARC-AGI-3 CPU	G0: own FX explorer clears >= 25% of frontier level starts at 40k actions	FAIL G0
13	P1B-DOTS3	2026-10-04	ARC-AGI-2	Dots3-Note IQ2_XXS L4x4 probe with forced 30k thinking budget; ub512 vs ub4; 24 tasks pass@1 (P1 v1	FAIL (infra: build)

#	Ledger id	Date	Track	Hypothesis (short)	Verdict
14	B13-X6D16-res	2026-10-05	ARC-AGI-3 serving	16 streams + mamba 80 + HiCache (D38)	FAIL (D38 prefill criterion; no job-level gain)
15	K1-WASTE	2026-10-06	ARC-AGI-3 harness	Recovering wasted decode (answer forcing at the token cap, filling idle slots) is worth >= 10% more	KILLED (CPU)
16	K2-DRAIN	2026-10-06	ARC-AGI-3 harness	History drains hurt: keeping more history (drain floor 58K -> 81.5K) raises levels	KILLED/PARKED (CPU)
17	K5-EFFIC	2026-10-07	ARC-AGI-3 harness	The action-efficiency term (s_i = min(1.15,(h/a)^2)) costs a large share of our score	KILLED (CPU)
18	B15C-FB25-res	2026-10-07	ARC-AGI-3 serving	vLLM mixed NVFP4-FP8 at 12 streams beats B12 at full budget	KILL
19	K-SURPRISE	2026-10-09	ARC-AGI-3 harness	An automatic surprise-halt (stop a batch after an action whose interior diff is > max(20, 3x that ac	KILLED (CPU)
20	HT-FB25	2026-10-10	ARC-AGI-3 harness	Fresh-eyes analysts at every SDT fire (2 fresh-context diagnoses through goal / controls lenses appe	PARK by D67 levels (108; 104-111) but HT_VALID FAIL: analysts answered 0 of 28 calls (D70)
21	SDT-FB25-res	2026-10-10	ARC-AGI-3 harness	SDT pair readout (D58/D60)	KILL (D64)
22	HT-FB25-res	2026-10-11	ARC-AGI-3 harness	HT readout (D67 gates, n = 1)	PARK (levels) / HT INERT -> not replicated as built (D70)
23	TC27-FB25-res	2026-10-11	ARC-AGI-3 engine	TC27 readout (D65)	KILL (D66)
24	D27-FB25-res	2026-10-11	ARC-AGI-3 engine	D27 readout (D65)	KILL (D66)
25	G4-D25	2026-10-11	ARC-AGI-3 engine	Gemma-4-31B-it (Google QAT W4A16 + QAT assistant MTP) on the B12 harness vs the B12-D25 pair, 25 gam	KILL (D69)
26	G4-D25-res	2026-10-11	ARC-AGI-3 engine	G4 readout (D68 gates)	KILL (invalid: 12/25 fed < 15; levels 5 < 40) (D69)
27	HT2-FB25	2026-10-11	ARC-AGI-3 stall	Fresh-eyes analysts that answer directly (no thinking) add levels on SDT stalls (D71)	PARK (D74)
28	X7B-FB25-run	2026-10-11	ARC-AGI-3 memory	Drain-aligned rolling summaries add levels at no prefill cost (D72; build D50)	KILL (D74)
29	B28-TC-FB25-res	2026-10-28	ARC-AGI-3 model	ThinkingCap W4A16 view (D78)	KILL (D79)
30	B29-OCC-FB25-res	2026-10-29	ARC-AGI-3 occupancy	fill client-time gaps with 12 active streams, server maxreq stays 10 (D80)	KILL (D84)
31	B31-SPEC2-FB25-res	2026-10-29	ARC-AGI-3 engine	fewer draft tokens raise aggregate tok/s at batch 9-10	INVALID (24/25 played: lf52 never admitted) + mechanism KILL
32	B31-SPEC1-FB25-res	2026-10-29	ARC-AGI-3 engine	fewer draft tokens raise aggregate tok/s at batch 9-10	INVALID (23/25 played: lf52 + su15 never admitted) + mechanism KILL

Row 30 also played only 24 of 25 games (lf52 never admitted, D84). Like rows 31 and 32 it is left out of run-level comparisons; its played attempts stay in the level-attempt fit (Appendix B, "partial").

Appendix B. All 39 development runs

#	Run file	Date	Games	Levels cleared	Score	Run minutes	Group	Used in
1	b1_franzen	2026-10-01	10	44 of 72	44.4	25		
2	b1c_ctrl	2026-10-01	10	43 of 72	41.7	25		
3	b2t_spec4	2026-10-01	1 of 10	0 of 72	0.0	20	partial: tu93, lp85, vc33, r11l, sc25, sb26, re86, ft09, tr87 never started	
4	b3_x2	2026-10-01	10	41 of 72	40.1	25		

#	Run file	Date	Games	Levels cleared	Score	Run minutes	Group	Used in
5	b5_x2ho	2026-10-01	15	20 of 111	7.1	40		
6	b5c_ctrlho	2026-10-01	15	22 of 111	7.3	39		
7	b4_x6	2026-10-02	10	45 of 72	46.9	25		
8	b7_x6ho	2026-10-02	15	28 of 111	13.3	40		
9	b7c_ctrlho2	2026-10-02	15	24 of 111	10.2	39		
10	f1_forklab	2026-10-02	240 of 256	593 of 1800	18.0	102	fork replay: 32 saved states x 4 arms x 2; partial: cd82, cd82, cd82, cd82, g50t, g50t, g50t, g50t, dc22, dc22, dc22, dc22, cd82, cd82, cd82, cd82 never started	
11	b10_x9	2026-10-03	10	23 of 72	12.8	25		
12	b8_x7	2026-10-03	10	32 of 72	25.7	25		
13	b9_x8	2026-10-03	10	46 of 72	44.8	25		
14	b12d25	2026-10-05	17 of 25	43 of 183	14.6	42	partial: wa30, ar25, sb26, re86, s5i5, ls20, su15, g50t never started	
15	b13_x6d16	2026-10-05	21 of 25	46 of 183	14.3	41	partial: sb26, re86, s5i5, su15 never started	
16	b12d25_r2	2026-10-06	17 of 25	45 of 183	18.4	41	partial: lf52, ar25, sb26, re86, s5i5, ls20, su15, g50t never started	
17	b15_vllm	2026-10-06	20 of 25	59 of 183	23.6	41	partial: ar25, sb26, re86, s5i5, su15 never started	
18	b15_vllm_r2	2026-10-06	20 of 25	48 of 183	16.9	41	partial: ar25, sb26, re86, s5i5, su15 never started	
19	b12_fb25	2026-10-07	25	106 of 183	48.3	122	control	C S F T B
20	b12_len	2026-10-07	25	110 of 183	50.7	123	B12 variant	F T B
21	b15c_fb25	2026-10-07	25	103 of 183	42.8	122	B12 variant	F T B
22	b12_sdt	2026-10-10	25	101 of 183	44.8	123	B12 variant	F T B
23	b12_sdt_r2	2026-10-10	25	106 of 183	48.4	123	B12 variant	F T B
24	b23_d27	2026-10-10	25	46 of 183	10.2	122		
25	b23_tc27	2026-10-10	25	48 of 183	10.1	122		
26	b12_ht	2026-10-11	25	108 of 183	52.2	123	B12 variant	F T B
27	b12_ht2	2026-10-11	25	110 of 183	49.8	122	B12 variant	F T B
28	b12_x7b	2026-10-11	25	100 of 183	47.4	123	B12 variant	F T B
29	b24_g4	2026-10-11	12 of 25	5 of 183	0.2	41	partial: wa30, lf52, r11l, ar25, sb26, cd82, re86, s5i5, ls20, ft09, su15, tr87, g50t never started	
30	b28_ctl_fb25	2026-10-28	25	95 of 183	42.6	122	control	C S F T B
31	b28_tc_fb25	2026-10-28	25	86 of 183	30.3	123	B12 variant (ThinkingCap, KILL)	T B

#	Run file	Date	Games	Levels cleared	Score	Run minutes	Group	Used in
32	b29_occ_fb25	2026-10-29	24 of 25	103 of 183	48.7	122	B12 variant; partial: lf52 never started	T
33	b30_safe	2026-10-29	1	4 of 6	47.6	5		
34	b31_spec1_fb25	2026-10-29	23 of 25	100 of 183	45.9	123	B12 variant; partial: lf52, su15 never started	T
35	b31_spec2_fb25	2026-10-29	24 of 25	115 of 183	54.6	122	serving-only change (spec2); partial: lf52 never started	T
36	b32_ctl_r2	2026-10-29	25	103 of 183	47.8	123	control	C S F T B
37	b32_spec2_r2	2026-10-29	25	97 of 183	42.8	123	serving-only change (spec2)	S F T B
38	b32_ctl_r3	2026-10-30	25	107 of 183	50.4	123	control	C S F T B
39	b32_spec2_r3	2026-10-30	25	107 of 183	49.2	122	serving-only change (spec2)	S F T B
	Later runs (after the 4 Nov analysis freeze; not in the counts above)							
40	v5_fb50	2026-11-04	25	141 of 183	63.8	243	control at 2x play time (V5)	
41	t1_fb25_nospec	2026-11-04	23 of 25	86 of 183	38.9	124	T1: speculative decoding off (V10); partial: wa30, su15 never started	
42	t1r_fb25_nospec	2026-11-04	24 of 25	82 of 183	35.8	124	T1 rerun, same notebook (V10); partial: su15 never started	
43	t1c_fb25_ctl	2026-11-04	25	112 of 183	51.7	122	control, same day as T1 (V10)	
44	v11_l1guard	2026-11-05	23 of 25	103 of 183	47.7	122	level-1 games keep their slot (V11); partial: lf52, su15 never started	

39 runs, 1,022 game runs. Score is local (the run's own games), not a leaderboard score. Games shows games played of games scheduled when they differ. Used in: C = crack-model fit and the V5 baseline (4 controls); S = same-agent noise: Fig. 3, the level SD and the A/A spread of b (6 runs); F = family variance and efficiency cost, Sections 3.3 and 4.3 (13 runs); T = the level-attempt fit of b, Section 3.5 (every played attempt of 17 runs); B = run-level comparisons of b, Section 4.5 (14 runs).

Appendix C. The agent and its slot rule

What the agent is. B12 is dfranzen's public Milestone 2 notebook on Tufa Labs' Duck harness (Jeroen Cottaar) with two changes (Section 3.1): the solved-level ledger (X6), in which the rule the model wrote for each solved level stays in the prompt for the whole game, and the slot-priority rule D' from shiiin9's public notebook (Form D1). We credit all three; the harness and nearly all of the agent are theirs.

One turn. Each game runs in its own thread with its own conversation. Each turn the model sees the current frame as an image with the recent history, thinks, and answers with a tool call: an action, or Python code whose functions persist for the rest of the game. Settings: a 128k-token context window, at most 12k generated tokens per turn, temperature 0.7. When the history outgrows the window (or 150 turns), the harness drops the oldest turns in one large cut (about 58k tokens; a drain) rather than a few at a time, because any cut at the front invalidates the server's cached prefix.

Slots. The server runs 10 generation streams, and a gate lets at most 10 games hold one. A game keeps its slot while its cached prefix is intact. After a drain its next request must be prefilled anyway, so that is when it hands its slot back and competes again with a priority computed from its current state; the highest priority in the queue gets the slot, and the game that handed over may win it straight back. Games that have not started wait in the same queue. For a game on level l of N, with a actions and t generated tokens spent on that level:

```
priority = A x M x C + B x phi
A    = (1 + 0.5 (l - 1)) x norm(N) x (300 / (300 + a))^2.5
norm = 55 / (N' (N' + 1) / 2), N' = N clipped to 6..10
M    = clip((30,000 / pace)^0.4, 0.25, 4); 1 before the first clear
C    = 0.1 max(0.1, 1 - a / 115) + 0.9 max(0.1, 1 - t / T)
T    = 225,000 x clip(pace / 30,000, 0.5, 2)^0.5; pace = 30,000 before the first clear
B    = 16, 14, 10 or 0 with 3 or more, 2, 1 or 0 levels left
phi  = 1, falling linearly to 0 over the last 40% of the run
```

A rewards depth and few actions (norm is the weight of a level in the game's score), M rewards a fast pace (pace: the game's mean tokens per cleared level), C is the patience on the current level, B is a bonus for levels left, and phi fades that bonus out at the end (the endgame of Appendix D).

What the rule does. A game that has not started has a = t = 0, so its priority is norm(N) + 16 (every public game has at least 6 levels): games with fewer levels start first, and the longest games start last. A game stuck on its level loses priority with every token and action. At the median state of the 14 shelved 0-level game runs of Section 4.3 (51 actions and 77k tokens on level 1), A x M x C is 0.44 x norm(N) against norm(N) for a game that has not started, so at its next drain the stuck game yields its slot to almost any waiting game, and later to games that keep clearing levels. Over the last 40% of the run the headroom bonus fades out and the order is A x M x C alone, which favours deep levels and fast games.

Long games start last. The slot rule ranks the games still waiting for their first turn by the value of their first level, which is highest in games with few levels (above). Ten games start at once and the others roughly in order of their number of levels, so lf52 (10 levels) starts at minute 89.3 of 121 on average. The 9 games that start after minute 30 clear 0.48 of their levels against 0.65 for the rest, but the gap is mostly the games themselves: within a game the start minute varies little between runs (SD 9.1 minutes), and a later start made no detectable difference (-0.02 levels per 10 minutes, permutation p = 0.767). Late starts matter when serving slows down or slots are held longer: then the last games never start (T1 and V11, Section 4.4).

Appendix D. The sealed tests in full

The private ARC-AGI-3 leaderboard (sealed 2 November 2026). Before Kaggle published the private board, we wrote down what the lottery model (H) predicts and what a low-noise alternative (S) predicts. In H, a run's score on each half of the hidden games is the agent's skill plus independent run noise with an SD of about 2.9 points, and a team's public score is its best run, so teams with more entries hold more tickets. In S, the noise is small (SD 1.0) and skill grows by 0.5 points per doubling of entries. On the public board alone, S and H with a noise SD of 2.3 fit about equally well (H with our measured 2.87 fits somewhat worse), so only the private board can decide. The predictions concern the teams with final public scores from 20 to just under 40 (the crowd C; 2,259 teams when we fitted the model):

- P1: rank correlation of public and private scores within C: H 0.39 (0.28 to 0.57), S 0.90; H fails above 0.70.
- P2: share of C's public top 10% that stays in C's private top 10%: H 30% (20% to 42%), S 71%; chance is 10%; H fails above 55%.
- P3: private-score gap between teams with 64 or more entries and teams with one (5.42 points on the public board): H 0.9 (0.0 to 1.7), S 3.6; H fails above 2.5.
- P4: how much more C's public top 10% falls from public to private than C as a whole: H 2.7 points (1.9 to 3.3), S 0.70; H fails below 1.3.
- P5: rank correlation among teams scoring 40 or more: H 0.89 (0.78 to 0.96); H fails below 0.60. Noise reorders the crowd, not the top.
- P6: our own runs: the private SD of our 8 B12-family submissions (the five B12 runs, B28 and both B30 runs in Table 1) is 1.7 to 4.2; our best public draw (33.37) falls back to about 28 (22.5 to 34.5) after the board-wide shift.

The decision rule was fixed in the same file: H passes if none of P1 to P4 crosses its line and at least three fall in their ranges; H fails if two or more cross. Two outcomes are covered by neither clause: exactly one crossing, and no crossing with fewer than three in range. A dated note kept apart from the sealed file (6 November) says both will be reported as undecided, with every value shown. The file's SHA-256 (2c5b49a9...) was posted on Kaggle (discussion 755123) and

time-stamped in Bitcoin block 974295. Outcome: pending. The hosts had not published the private leaderboard when this version was written; score_sealed.py applies the rule above when they do, whatever the result. Anyone can run it from the repo with three CSV files (the final public board, the private board and our submissions list): `python3 analysis/s37/score_sealed.py PUBLIC.csv PRIVATE.csv OURS.csv`.

Twice the play time (V5, sealed 4 November 2026). The crack model predicts that doubling the play time per game (242 instead of 121 minutes, everything else byte-identical) raises the level total from 102.75 (the mean of the four 1x controls) to 127 (90% interval 110 to 145), with 13 games completed (10 to 16) instead of 7 to 8. The sealed rule: a total from 110 to 145 passes the model; 109 or less fails it toward "stuck" (the games that hit the 121-minute limit were mostly at walls); 146 or more fails it toward "out of time" (progress kept coming at close to the first-half rate). The run was valid: all 25 games played, no tracebacks, 8.49M generated tokens. It cleared 141 levels, inside the sealed interval, so by the sealed rule the crack model passes (Fig. 5). The three secondary checks also landed inside their ranges: 11 games completed (predicted 13, 10 to 16), 105 levels by minute 121 (about 103, 93 to 113), and a rank correlation of 0.48 between predicted and actual per-game gains over the 19 open games (sealed median 0.22, 90% -0.23 to 0.54). The pass is not comfortable. The total is 14 levels above the point prediction, five below the "out of time" line, and close to the log-linear alternative stated in the same file (about 140); saturation (about 103) and linear growth (150 to 175) are ruled out. The sealed file was pushed to our GitHub repository at 10:18 UTC on 4 November, ten minutes before the run started (10:28 UTC); its public OpenTimestamps proof is anchored in Bitcoin block 974519 (10:52 UTC), 24 minutes into the run and 3.6 hours before it finished (14:31 UTC).

Afterwards we checked where the extra levels came from (exploratory). The harness's slot-priority rule has an endgame: over the last 40% of each game's budget it fades out a bonus for games with levels left, which moves streams to games close to finishing. At 121 minutes per game the endgame starts at minute 72.6; at 242 minutes, at minute 145.2. The sealed calculation continued the control runs' end states, which already include their endgame: 7 to 8 games finished and the rest on old attempts (mean 77k to 100k tokens). V5 reached minute 121 before its endgame, with 3 games finished and younger attempts (mean 53k). From V5's own state the sealed calculation gives 42.9 further levels instead of 24.4 to 27.1, and with the tokens each game actually generated afterwards the model expects 38.5, against 36 observed. Between minutes 72.6 and 121 the six games stuck at 1x (at most 1.5 levels on average in the controls) received 0.74M tokens in V5 and went from 6 to 13 levels; in the control runs' endgame they received 0.21M to 0.52M and gained 1 to 3. A second opinion committed after the seal but before the result, a simulator in which finished games hand their streams to the others, had predicted 135 to 139; it has no endgame either. The one sign that walls are less permanent than the model assumes is in the long tail: attempts that passed 150k tokens cracked 6 times against 2.9 expected from the fitted hazard (Poisson $p = 0.073$).

A later simulation sizes the endgame's share (exploratory; written after the V5 result and committed before the V11 readout). It plays 100 worlds of the crack model with persistent level difficulty and walls (H2, Section 3.4) through the harness's slot rule (Appendix C) on 10 slots, with and without the endgame. It comes close on both budgets: 106.5 levels at 121 minutes (observed 102.75) and 139.2 at 242 (V5: 141). The endgame barely changes what doubling buys (32.7 levels with it, 31.9 without), and a 121-minute run continued to 242 minutes ends at the same total as a 242-minute run. What it changes is the sealed calculation's input. Applied to 4 simulated control runs per world, the sealed calculation predicts a gain of 24.5 levels (from the real controls: 24.4 to 27.1), 8.3 short of the simulated gain; from control runs simulated without the endgame it falls 1.5 short. The difference, 6.8 levels (SE 0.8; 5.9 and 10.0 if a context trim, which hands the slot back, comes every 80k or 40k generated tokens instead of 58k), is about half of the 14 between 127 and 141. V5 ran another 5.7 above 127 plus the simulated shortfall, about one run-to-run SD. The simulator misses one check: from its 242-minute runs' state at minute 121 the sealed calculation falls 5.0 short, while from V5's state it gave 42.9 against 36 observed.

Less token supply (V10, sealed 4 November 2026). If the agent is limited by tokens, a run that generates fewer tokens on the same clock should play like a control run stopped early. We sealed this as a table before launch (committed at 16:42 UTC; Bitcoin block 974554): with speculative decoding off (T1: the same notebook, games and 121-minute budget), the level total should follow the controls' mean clock curve at minute $r \times 121$, where r is T1's generated tokens over the controls' mean. A control run on the same day (T1c) checked for drift (92.7 to 112.8 levels, tokens within 5%). The file's validity rule asks for all 25 games played, and an invalid run is rerun once. Both T1 runs were invalid. They

generated 0.72 and 0.73 of the control tokens, and games were left that never started (2 in the first run, 1 in the rerun). The harness plays at most 10 games at a time, and every game's budget counts from the start of the run, so a game that waits for a turn loses time: in the four controls and T1c the last game started 69 to 99 minutes in. With slower serving, the last games never got a turn. By the sealed rule the supply prediction is untested. Exploratory only: the two totals, 86 and 82 levels, lie inside the sealed bands at their measured supply (86.7, 77.6 to 96.0; and 87.2, 78.1 to 96.4). T1c was valid and showed no drift: 112 levels with 0.997 of the control tokens. It is the highest of the seven same-agent runs and raises their SD from 5.3 to 6.0 (the simulator gives 5.6).

A first-level guard (V11, sealed 5 November 2026). If most zero-level games are shelved rather than walled (Section 4.3), keeping games on level 1 from being shelved should remove most zeros. We sealed this before launch (committed at 14:32 UTC; kernel pushed at 14:40 UTC; Bitcoin block 974681, 15:34 UTC). V11 is the control notebook with one change: a game that has started on level 1 gets top slot priority until it clears the level or has spent 150k tokens on it. The sealed rule counts the games that start and end with 0 levels: PASS if at most 1, FAIL if 2 or more. The seven reference runs had 1 to 4 (mean 2.43), so with no effect a pass has about a 1 in 7 chance. V11 passed with 1 of the 23 games that started: tn36, which had spent 161k tokens on level 1, past the guard's limit. All six games with zeros in the reference runs cleared level 1, sk48 (a zero in 6 of the 7) included. The run was partial, though: 2 games never started (lf52 and su15), which the sealed file reports instead of rerunning; counting them as zeros gives 3. The total, 103 levels, matches the reference mean (103.9, SD 6.0), with 0.86 of the control tokens. One run is a weak test. In the scheduler simulation above, committed before the readout, the guard cut zero-level games from 3.25 to 2.21 per run and raised the games that never start from 0.20 to 0.63. Rerun after the readout at this run's token supply (exploratory), the simulation leaves 2 or more games unstarted in 18% of runs without the guard and in 59% with it (9% with the guard at full supply), so the guard and the lower supply probably both played a part; it also expects fewer levels at that supply (90.5) than V11 cleared.

The private ARC-AGI-2 leaderboard (V9, sealed 4 November 2026). After the ARC-AGI-2 close and before Kaggle published its private board, we refitted the lottery model to the plateau of Section 4.8 and sealed what it predicts. A team's public score is its skill plus the best of its noisy draws; a fitted share of its entries (30%) count as draws, and skill may rise with the number of entries (fitted: 0.3 points per doubling). The public board's own fit is best at a noise SD of 1.0, but that fit leans on the assumed shapes of skill and entry counts, so the primary prediction uses the SD measured on our own reruns: 0.6 points per half-board, the pooled SD of our same-code draws. The predictions concern the 2,133 teams with public scores from 27 to just under 35:

- P1: rank correlation of public and private scores: 0.82 (0.65 to 0.91).
- P2: share of the plateau's public top 10% that stays in its private top 10%: 60% (43% to 73%); chance is 10%.
- P3: private-score gap between teams with 64 or more entries and teams with one (2.87 points on the public board): 2.0 (1.0 to 2.3).
- P4: how much more the public top 10% falls from public to private than the plateau as a whole: 0.48 points (0.29 to 0.88).
- P5: rank correlation among the 106 teams scoring 40 or more: about 0.99; the model fails below 0.90.

Three named alternatives carry their own values for P1 to P4: a noise SD of 0.81 (all seven of our draws) gives 0.69, 48%, 1.6 and 0.72; an SD of 1.0 (the public board's best fit) gives 0.52, 37%, 1.2 and 1.08; low noise (SD 0.3, with the entry-count gap mostly skill) gives 0.95, 79%, 2.4 and 0.17. The primary passes if at least three of P1 to P4 fall in their ranges and fails if two or more fall outside on the same side. A second test does not depend on the chosen SD: each of P1 to P4 implies an SD, and the model's structure passes if the largest implied SD is at most 1.5 times the smallest. The file's SHA-256 (1fffeeef...) was posted on Kaggle as a comment in discussion 755123 and time-stamped with OpenTimestamps (Bitcoin block 974554, 17:27 UTC). Outcome: pending, as for ARC-AGI-3; score_v9.py applies the rule above when the private board is published (`python3 analysis/s37/score_v9.py PUBLIC.csv PRIVATE.csv`).

Appendix E. More detail on Sections 4.3 and 4.6 to 4.8

Fewer walls than the four-run fit says (Section 4.3). Refitted on all eight runs of this agent (the seven valid 121-minute runs and V5; 1,003 attempts), the wall share falls from 6% to 4% of levels (median wait 24.2k tokens, s 1.10). The refit has seen V5, so this is a consistency check, not a test: it expects 101.2 levels at 121 minutes and 140.2 at twice the budget (observed: 102.75 on average and 141). It calls 28% of the controls' open attempts walls. Fourteen levels were reached but never cleared in any of the eight runs, and the refit makes 9 of them walls with probability 0.5 or more. The clearest is level 5 of dc22: reached in 6 runs and left open after 880k tokens in total (218k in V5 alone). If those nine are fixed walls, unlimited play caps this agent at 154 of the 183 public levels. Another 40 levels were cleared in some runs and not in others.

Inside the walls (Section 4.3). This check is exploratory, written after outside reviewers asked what the walls are, and it uses the action logs only, because the agent's own notes are not in our data. For each of the 9 likely walls we compared the actions the agent took on the level with the human baseline h that the scorer uses. On the held-out levels it clears, the agent needs a median of 0.72 h actions at 756 generated tokens per action; on the walls it spends 1,597 tokens per action (median over the 9), about twice as long per move. The walls are of two kinds. In 6 of the 9, some run took at least 0.72 h actions and still found no way through: g50t level 5, for example, got 244 actions, 2.54 times the human baseline. These look like failures to find the goal or the mechanic, not a shortage of moves. In the other 3, no run came close to the actions a person needs: dc22 level 5 asks people for 324 actions, the agent took at most 102, and at its 1,950 tokens per action there, reaching 0.72 h would take about 456k tokens, roughly twice the 218k it got in the doubled run. Walls are not simply the longest levels: relative to the earlier levels of their game, their human baselines are only somewhat larger than those of the levels the agent clears (median ratio 1.98 against 1.31; AUC 0.56). Telling a wrong goal from a missing mechanic needs the agent's reasoning, which a rerun that keeps its notes, or a person playing the level, can supply.

Half of the per-game noise is the first level (Section 4.3). Levels are played in order, so a game that never clears level 1 scores nothing. Over the seven 121-minute runs, the split between 0 levels and at least one accounts for 49% of the summed per-game variance (bootstrap over runs, 90% interval 39% to 57%), almost all of it in four games (sc25, cd82, dc22 and g50t). These zeros are not walls. Of the 17 game runs that ended with 0 levels, 14 were shelved by the harness's slot rule. At most 10 games play at once; a game hands back its slot whenever its context is trimmed and then queues behind games with a higher priority, and its own priority falls with the tokens and actions spent on its current level (Appendix C). The shelved games had spent 56k to 132k tokens on level 1 (median 77k) and did not play again; 7 of them stopped by minute 20. In V5 all 6 of these games cleared level 1, after 40k to 350k tokens. A game's first crack therefore races the slot rule's patience as well as the clock, and that patience can run out early in the run. A sealed one-run test that protects games on level 1 from shelving passed (V11, Section 4.4): 1 of the 23 games that started ended with 0 levels, but 2 games never started.

How many runs it takes to see a real change (Section 4.6). On the leaderboard one submission has an SD of 2.87 points. To detect a true difference with power 0.8 (two-sided test at 0.05) takes 15 submissions per arm for a gain of 3 points and 6 for a gain of 5; one run per arm resolves only 11.4 points. These counts use the normal approximation; because the SD is estimated from the runs themselves, a t-test needs one more run per arm (16 and 7, simulated). The SD is itself uncertain: from five submissions its 90% interval is 1.86 to 6.81 points, so the 15 could be anywhere from 7 to 81. Resubmission alone buys points: the best of n identical submissions beats their mean by 1.62 (n = 2), 3.34 (5), 4.42 (10), 5.36 (20), 6.45 (50) and 7.20 points (100).

In development runs, the two numbers of Section 3.5 help. A 10% change in crack efficiency moves a run's level total by 4.4 levels and its crack cost b by 0.095. On the 6 valid same-agent runs, b has an A/A SD of 0.051 and the level total an SD of 5.3, so a b comparison needs 4.5 runs per arm against 22.6 for raw level totals, 5.1 times fewer. The factor rests on six runs and is fragile: leaving out one run at a time gives 2.8 to 65.1 times, so we claim only that b needs several times fewer runs, at least about 2.8. On the 8 valid variant runs, the two numbers predict the level difference from the control with r = 0.68. A CPU notebook (paper/notebook/how_many_runs.ipynb in our repository) computes these run counts, fits b for two arms from run logs (with our B1 and B12 runs as the example) and repeats the fit-half, predict-half check of Section 4.2 on a user's own runs.

Methods the model rules out (Section 4.7). We simulated schedulers under the fitted model before spending GPU time on them. Giving the 10 streams to the games with the highest current crack hazard gains +0.0% over equal sharing when games are alike, and -3.6% when their difficulty differs (log-median SD 0.8). An oracle that knows which games are

walled and skips them gains +16.9% and +13.4%. So a scheduler is worth building only with a wall detector. We then tried to predict, from the run logs (actions, clicks, tokens per call, level index, earlier levels' costs), which open attempts would not crack within the next 30k tokens, checking at 5k, 10k, 20k and 40k tokens into an attempt. Leave-one-game-out AUCs were 0.544, 0.491, 0.533 and 0.582, close to chance, so we did not build it.

The same lottery elsewhere (Section 4.8). ARC-AGI-2. Seven of our scored draws of one public notebook lineage have mean 29.66 and SD 0.81 (Table 2). They come from two notebooks, each resubmitted unchanged: A0 three times (mean 29.03, SD 0.86) and PP four times (mean 30.14, SD 0.34). On the final public ARC-AGI-2 board (3,627 teams), 2,133 teams (59%) sit on a plateau between 27 and 35. If these teams mostly resubmit near-identical public notebooks, each team's score is roughly the best of its n draws. Without fitting anything to the board, the best of n draws from our seven predicts a plateau mean of 29.65 for single-entry teams rising to 31.67 at 91 entries; observed means rise from 29.41 (282 teams with one entry) to 32.28 (64 teams with 64 or more) (Fig. 6). From 1 to 91 entries that is 2.01 of the observed 2.87 points (70%). Run luck alone, at the pooled SD of our unchanged resubmissions (0.61), gives 1.50 (52%); at PP's SD alone (0.34) it would be 0.84 (29%), and at A0's (0.86) 2.15 (75%). Inside the plateau, score and number of entries have a rank correlation of 0.55. The observed slope is steeper than an SD of 0.81 predicts and matches an SD of 1.0 at the top. The two notebooks' means differ by 1.11 points, so the plateau's spread mixes run luck with small code differences, and teams with more entries may also improve their code between entries. Run luck is about half of the effect, with wide bounds from only seven draws. Section 4.4 gives a sealed prediction of how this plateau reorders on the private board.

Other teams' harnesses. We fitted the crack model separately to six public runs by other teams. Four forks of the Duck harness show our median crack wait (20.2k to 28.1k tokens) but more walls (q from 0.22 to 0.37). Two runs show no walls at all (q = 0.00), one with a much slower median (65.1k) and one, on 10 games only, with a faster one (11.4k). In every case the per-run fit reproduces the run's level total within 1.3 levels.

One caveat on independence. dfranzen's public notebook plays four passes over the 25 games, not all at the same time. Their level totals (102, 106, 104 and 105) vary far less than independent games would make them (variance 2.9 against a null median of 40.9, p = 0.020). Some harnesses therefore couple passes, so our independent-game noise is an upper bound for them. Fewer level-1 zeros do not explain it: every pass started all 25 games and ended 1, 1, 2 and 3 of them with no level (sk48 in all four; analysis/s37/franzen_4pass_zeros.py), against 1 to 4 (mean 2.43) in our seven reference runs. We have not found the mechanism.

SWE-Bench-Verified (Section 4.8; exploratory, added 7 Nov 2026). analysis/s38/fetch_swe_runs.py reads the per-run harness reports of Bjarnason et al. from their public dataset (huggingface.co/datasets/ASSERT-KTH/agentic-evals-artifacts, CC BY 4.0) and keeps the ids of the resolved tasks: 12 configurations of 10 runs each over 500 tasks. analysis/s38/swe_pertask_luck.py takes each task's success rate p across the m runs of a configuration and predicts the variance of a run's number of resolved tasks as the sum over tasks of p(1

- p), times m/(m - 1) for the small sample. Pooled over the 12 configurations, the predicted variance is 86% of the observed one; redrawing runs under pure per-task luck with the same p (a parametric bootstrap, 4,000 draws, seed 0) gives a 90% band of 81% to 127% for this share. Per configuration the share is 51% to 200%, and the observed run SD is 0.75 to 1.76 points of resolution rate. In each configuration 100 to 331 tasks are never solved and 144 to 309 are solved in some runs only. The check uses final outcomes only; the trajectories in the same dataset also record tokens, which a crack-model fit in token time would need.